Prompt Inference Attack on Distributed Large Language Model Inference Frameworks

Xinjian Luo
xinjian.luo@mbzuai.ac.ae
Mohamed bin Zayed University of Artificial Intelligence
United Arab Emirates

Ting Yu
ting.yu@mbzuai.ac.ae
Mohamed bin Zayed University of Artificial Intelligence
United Arab Emirates

Xiaokui Xiao
xkxiao@nus.edu.sg
National University of Singapore
Singapore

Abstract

The inference process of modern large language models (LLMs) demands prohibitive computational resources, rendering them infeasible for deployment on consumer-grade devices. To address this limitation, recent studies propose distributed LLM inference frameworks, which employ split learning principles to enable collaborative LLM inference on resource-constrained hardware. However, distributing LLM layers across participants requires the transmission of intermediate outputs, which may introduce privacy risks to the original input prompts — a critical issue that has yet to be thoroughly explored in the literature.

In this paper, we rigorously examine the privacy vulnerabilities of distributed LLM inference frameworks by designing and evaluating three prompt inference attacks aimed at reconstructing input prompts from intermediate LLM outputs. These attacks are developed under various query and data constraints to reflect diverse real-world LLM service scenarios. Specifically, the first attack assumes an unlimited query budget and access to an auxiliary dataset sharing the same distribution as the target prompts. The second attack also leverages unlimited queries but uses an auxiliary dataset with a distribution differing from the target prompts. The third attack operates under the most restrictive scenario, with limited query budgets and no auxiliary dataset available. We evaluate these attacks on a range of LLMs, including state-of-the-art models such as Llama-3.2 and Phi-3.5, as well as widely-used models like GPT-2 and BERT for comparative analysis. Our experiments show that the first two attacks achieve reconstruction accuracies exceeding 90%, while the third achieves accuracies typically above 50%, even under stringent constraints. These findings highlight substantial privacy risks in distributed LLM inference frameworks, issuing a strong alert on their deployment in real-world applications. Additionally, our analysis uncovers distinct distributional properties of intermediate embeddings across LLM layers, providing valuable insights into the LLM inference process and the development of effective defense mechanisms for distributed LLM frameworks.

CCS Concepts

• **Security and privacy → Distributed systems security**.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '25, Taipei, Taiwan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
https://doi.org/10.1145/3719027.3744820

Keywords

LLM, Distributed Inference Framework, Prompt Inference

ACM Reference Format:
Xinjian Luo, Ting Yu, and Xiaokui Xiao. 2025. Prompt Inference Attack on Distributed Large Language Model Inference Frameworks. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), October 13–17, 2025, Taipei, Taiwan.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3719027.3744820

1 Introduction

Although demonstrating remarkable performance, recently released large language models (LLMs) require substantial GPU memory for inference, making them challenging to deploy on consumer-grade devices [4]. To address this limitation, recent studies [4, 5, 9, 19, 28, 50, 53] have proposed distributed LLM inference frameworks inspired by split learning [13], where LLM layers are distributed among multiple participants to enable collaborative inference. In this way, a portion of the inference process can be allowed to run locally on devices with limited computational resources. For example, Fig. 1 illustrates the Petals framework [4], which facilitates distributed LLM inference by introducing two types of modules: a client module that hosts the tokenizer, token embedding layer, and a few decoder layers, and a server module that manages intermediate decoder layers. During inference, intermediate sequence embeddings are exchanged between the client and server modules to enable a complete next-token prediction.

For these distributed LLM inference frameworks, a critical question arises: *to what extent can the privacy of input prompts be compromised via intermediate outputs?* This concern is particularly significant as users increasingly rely on LLMs for tasks involving confidential information, and the leakage of sensitive data to third parties could have serious consequences. For example, in digital healthcare, LLMs can analyze users' descriptions of symptoms and mobile sensor data for medical diagnosis [47]. Any leakage of such data in the prompts can compromise user privacy and significantly undermine trust in related applications. Similarly, LLMs are widely applied in business analytics [36], where input prompts often include proprietary information such as customer reviews and sales data. Leakage of such proprietary information could result in reputation damage and financial losses for companies. While some developers of distributed LLM frameworks acknowledge potential privacy risks associated with the transmission of intermediate results [4], they fail to provide definitive answers regarding *how and to what extent* private input prompts could be leaked. To bridge this critical gap, we propose inference attacks to systematically reconstruct private prompts from intermediate outputs in distributed

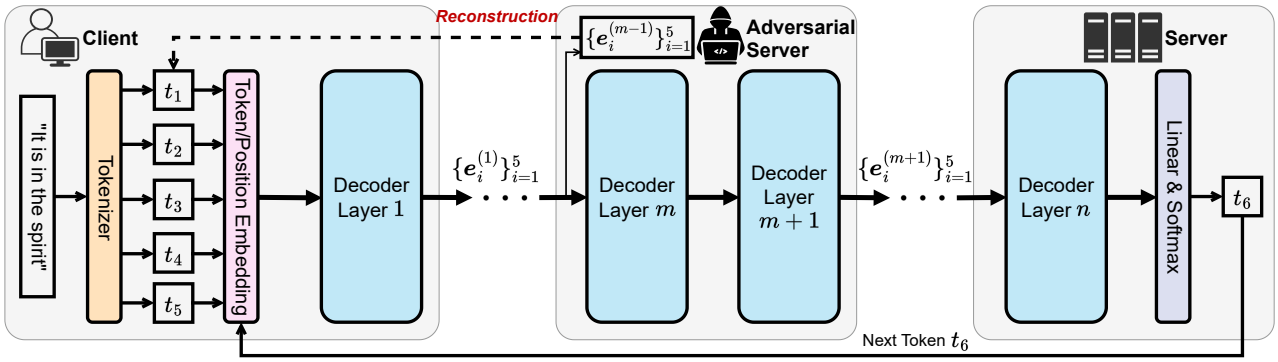


Figure 1: The overview of distributed LLM inference for next-token prediction.

LLM frameworks, offering an in-depth exploration of this under-explored vulnerability.

While conceptually related to input inference attacks in split learning [39, 54], our prompt inference task presents two critical differences that limit the applicability of existing split learning attack methods. First, split learning attacks focus on reconstructing image data, whereas attacks on LLMs target text data. Second, split learning attacks operate during the training phase and typically require training a shadow model to simulate layers outside the adversary's control. In contrast, our attacks are designed for the inference phase, where only one-time intermediate outputs are available for reconstructing each prompt, and training a shadow model for LLMs is impractical due to the prohibitive computational demands posed by the much larger size of LLMs compared to image classification models. Sentence embedding inversion attacks [22, 35, 38, 44] also share some conceptual overlap with our task. However, these methods target relatively small sentence embedding models and rely on transformer-based attack models for reconstructing sentences from embeddings. This approach suffers from two limitations. First, the length of input sentence embeddings for attack models is typically constrained (e.g., < 15 in [12]) because training transformer-based models for long contexts requires substantial amounts of data and computational resources — resources that most adversaries lack. Second, these reconstruction methods could be context-dependent, where an error in reconstructing a word may propagate and severely degrade the reconstruction accuracy for subsequent words, limiting the overall effectiveness of reconstruction. In this paper, we aim to develop novel attacks on LLMs that can overcome the aforementioned limitations. To achieve our attack targets, new insights into the workflow of LLM inference need to be developed.

To design effective attacks on distributed LLM frameworks, we analyze the distribution of sequence embeddings across different layers of LLMs. Our key observation reveals that intermediate embeddings output by different LLM layers typically form distinct clusters in the embedding space. Building on this insight, we propose three types of attacks by progressively tightening constraints on query budgets and auxiliary data, which can reflect diverse real-world application scenarios for distributed LLM inference and enable a comprehensive assessment of their privacy risks. Specifically, the first adversary operates with unlimited query budgets and auxiliary data following the same distribution as the target prompts. Here, instead of taking the prompt reconstruction task as a sentence generation problem [22, 34, 35], we reformulate it as a classification problem and leverage a trained classifier for prompt reconstruction. The second adversary, while also having unlimited query budgets, assumes auxiliary data with arbitrary distributions. For this case, we analyze the influence of sequence length and word constitution on intermediate embeddings and introduce a sequence embedding synthesis approach to augment query data for classifier training. The third adversary faces the most restrictive scenario: limited query budgets and no auxiliary data. In response, we design a three-phase reconstruction framework that first utilizes nearest neighbor search and semi-supervised learning for identifying key concepts of input prompts, and then employ a constrained beam search to reconstruct finer details.

Compared to related studies [22, 35, 38, 39, 44, 54], our attacks are lightweight (no shadow model training required), free from length constraints, and context-independent (errors in word reconstruction hardly propagate). Moreover, our methods are adaptable across different LLM architectures, making them LLM-independent. Experiments conducted on a variety of LLMs, including Llama-3.2 [31], Phi-3.5 [32], GPT-2 [6], and BERT [8], demonstrate reconstruction accuracies exceeding 90% for the first two attacks and reaching larger than 50% in most cases for the third attack. Notably, our results show that the embeddings in early LLM layers are more vulnerable to privacy breaches than those in later layers. This difference is mainly attributed to the distributional variations of sequence embeddings across the LLM inference pipeline. We believe these new findings will provide valuable insights into LLM inference mechanisms and the development of robust defense mechanisms for distributed LLM inference frameworks. To the best of our knowledge, this is the first work that develops attacks targeting distributed LLMs. Our contributions are summarized as follows:

- We uncover clustering patterns in sequence embeddings across LLM layers and leverage this to design three prompt inference attacks under varying constraints. Our attacks are lightweight, free of length constraints, context-independent, and LLM-independent.
- We perform experiments on well-recognized LLMs including Llama-3.2, Phi-3.5, GPT-2, and BERT. The results demonstrate larger than 90% reconstruction accuracies in most attack settings. Our results also reveal the differences in susceptibility along the LLM inference pipeline.
- We underscore the significant privacy risks in distributed LLM inference frameworks. We believe that our new findings on the LLM embedding distributions not only guide

the development of future defenses but also offer valuable insights into LLM inference mechanisms.

2 Preliminaries

In this section, we first outline the process of LLM inference and then present the general workflow of distributed LLM frameworks.

2.1 Decoder-Only LLMs

The Transformer architecture [46] was originally proposed to increase training parallelization, which relies solely on attention mechanisms to capture the dependencies between input and output sequences. Due to its scalability and efficiency, Transformer has become the backbone of modern LLMs. Note that the original Transformer was designed for machine translation with an encoder-decoder structure. Since the decoder component of Transformers is more suitable for text generation tasks, it has emerged as the primary architecture for state-of-the-art LLMs. Examples include GPT-2 [6], Phi-3.5 [32], and Llama-3.2 [31], collectively known as *decoder-only* transformer architectures.

The primary task for decoder-only LLMs is *next-token prediction*, the basis for their pre-training. Other tasks, such as semantic similarity [45] and text classification [10], are typically implemented by modifying the final linear layer while retaining the decoder structure used for next-token prediction. Fig. 1 shows a typical forward pass in an LLM during next-token prediction. An input prompt S is tokenized into a sequence of tokens $T = \{t_i\}_{i=1}^{l}$ with $t_i \in T_{\text{all}} = \{0, 1, \cdots, |T_{\text{all}}| - 1\}$, where T_{all} represents the set of all unique token IDs, and l is the sequence length. For example, in GPT-2, the input phrase "It is in the spirit" is tokenized as $T = \{1026, 318, 287, 262, 4437\}$ with $|T_{\text{all}}| = 50257$. These tokens are then passed to an embedding layer, producing token embeddings $B = \{\boldsymbol{b}_i\}_{i=1}^{l}$ with $\boldsymbol{b}_i \in \mathbb{R}^d$, where d denotes a pre-defined model dimension (e.g., $d = 1280$ for GPT-2). The token embedding layer essentially acts as a dictionary, mapping token IDs in T_{all} to their corresponding embeddings.

The embeddings B are then processed through a sequence of decoder layers, generating intermediate embeddings $E^{(m)} = \{\boldsymbol{e}_i^{(m)}\}_{i=1}^{l}$, where $\boldsymbol{e}_i^{(m)} \in \mathbb{R}^d$ and m denotes the decoder layer index. Specifically, each intermediate embedding $\boldsymbol{e}_i^{(m)}$ is computed as $\boldsymbol{e}_i^{(m)} = f_{1\cdots m}(\boldsymbol{b}_1, \cdots, \boldsymbol{b}_i)$ with $f_{1\cdots m}$ denoting the first m decoder layers, i.e., the generation of the i-th intermediate embedding is conditioned on all tokens proceeding token t_i. The embedding $\boldsymbol{e}_i^{(m)}$ can also be viewed as a *sequence embedding* for the token sequence $\{t_1, \cdots, t_i\}$ at layer m . After processing through all n decoder layers, the last embedding $\boldsymbol{e}_l^{(n)}$ of the final embedding outputs $E^{(n)}$ is passed to a linear layer, which predicts the next token t_{l+1}. The token t_{l+1} will then be appended to T for subsequent predictions. Thus, a complete forward pass of LLMs can be represented as:

$$S \xrightarrow{\text{tokenizer}} T \xrightarrow{\text{embedding}} B \xrightarrow{f_1 \cdots f_n} E^{(n)} \rightarrow e_l^{(n)} \xrightarrow{\text{linear layer}} t_{l+1}.$$

2.2 Distributed LLM Inference

The performance of LLMs is well known to scale with model size, leading state-of-the-art LLMs to consist of tens or even hundreds of billions of parameters. Consequently, running such massive models requires high-end hardware resources, which are inaccessible to most researchers and practitioners. To address this limitation, distributed LLM inference frameworks [4, 5, 9, 50] have emerged as a practical solution, enabling LLM inference on consumer-grade devices. The core idea behind distributed LLM inference is to partition the model into smaller modules. Typically, the client module hosts the embedding layer and a few initial decoder layers, while the server module hosts the remaining intermediate decoder layers. During inference, the client initiates a query, processes the input through its hosted layers, and then transmits the intermediate outputs to the server modules. Fig. 1 illustrates this process in a distributed setup. By splitting the model in this way, each participant only needs to host a portion of the model parameters, making it feasible to perform LLM inference on low-end hardware. In addition, this collaborative framework allows participants to flexibly choose whether to host a client, a server, or both [4].

A significant concern in distributed LLM inference frameworks arises from the transmission of intermediate outputs between participants, which may expose sensitive information about the input prompt. While some developers [4] acknowledge potential privacy risks associated with sharing intermediate results, they fail to provide conclusive evidence or a comprehensive analysis of *how and to what extent* input prompt privacy may be compromised.

3 Problem Statement

In this section, we first introduce the system model illustrating how an adversary participates in distributed LLM inference and then discuss possible adversarial settings for prompt inference attacks.

3.1 System Model

We follow the settings of distributed LLM frameworks as described in [4, 5], where an adversary can host both a client module and a server module. Without loss of generality, we assume that the adversary's server module begins at the m-th decoder layer, as illustrated in Fig. 1. During the inference phase, a participant initializes a query by inputting a prompt $S^{\text{tgt}} \sim P_{\text{tgt}}$ into the client module. P_{tgt} denotes the distribution or domain of prompt S^{tgt}. S^{tgt} is also treated as the *target prompt* to be reconstructed by the adversary. The intermediate outputs will be continually processed to be $E^{\text{tgt}(m-1)}$ and then forwarded to the adversary's server module starting from layer m. Consequently, the adversary always has access to the intermediate results $E^{\text{tgt}(m-1)}$ generated for any query initiated by other participants. Since we focus on developing generalized attacks that can work with embeddings from any intermediate decoder layer, we will omit the specific layer index $m-1$ and refer the embeddings received by the adversary as E^{tgt} for simplicity. Similarly, the adversary can generate its own queries from auxiliary data $S^{\mathcal{A}} \sim P_{\mathcal{A}}$ using the client module and obtain the corresponding embeddings $E^{\mathcal{A}}$. The maximum number of tokens allowed in total queries is referred to as the *query budgets*.

3.2 Attack Model

In this paper, we focus on developing black-box, passive prompt inference attacks aiming at reconstructing the target prompts S^{tgt}

Table 1: Attack settings of our prompt inference attacks.

Adversary	Query Budgets	Auxiliary Data $\mathcal{S}^{\mathcal{A}}$	Data Distribution
$\mathcal{A}_1$	Unlimited	✓	$P_{\mathcal{A}} \approx P_{\text{tgt}}$
$\mathcal{A}_2$	Unlimited	✓	$P_{\mathcal{A}} \not\approx P_{\text{tgt}}$
$\mathcal{A}_3$	Limited	✗	/

initialized by other participants. The *black-box* setting [24, 25] assumes that the adversary has no knowledge of components beyond the client and server modules it hosts. The *passive* (or semi-honest) setting [17, 26] indicates that the adversary does not tamper with the LLM inference process but instead infers private prompts S^{tgt} solely from the received sequence embeddings E^{tgt}. In addition, we consider designing attacks under different constraints on query budgets and auxiliary data availability, as summarized in Table 1.

Specifically, $\mathcal{A}_1$ has unlimited query budgets and access to an auxiliary dataset $\mathcal{S}^{\mathcal{A}}$ sampled from the same distribution as the target prompt S^{tgt}, i.e., $\mathcal{S}^{\mathcal{A}} \sim P_{\mathcal{A}} \approx P_{\text{tgt}}$. This scenario reflects cases where both auxiliary and target data originate from the same domain, such as Wikipedia. The assumption of unlimited query budgets aligns with current distributed LLM inference frameworks [4, 5]. The second adversary $\mathcal{A}_2$ also has unlimited query budgets but operates under a mismatch in distributions between auxiliary data and target prompts, i.e., $P_{\mathcal{A}} \not\approx P_{\text{tgt}}$. This scenario indicates that the adversary has no knowledge of the domain of target prompts S^{tgt}. For example, while $S^{\mathcal{A}}$ could be sampled from Wikipedia, S^{tgt} may be sampled from Reddit [1]. The third adversary $\mathcal{A}_3$ operates under the most restrictive setting, with limited query budgets and no auxiliary data. Although limited query budgets are not explicitly discussed in prior distributed LLM inference studies [4, 5], they are relevant in similar LLM-related commercial scenarios, such as pay-as-you-go API services provided by OpenAI [37] and Google [1]. Under this setting, the objective is to maximize attack effectiveness with small query costs.

4 The Attack Methods

In this section, we introduce the proposed attack methods corresponding to the three adversaries introduced in Section 3.2.

4.1 The First Adversary $\mathcal{A}_1$

The first adversary operates with unlimited query budgets and an auxiliary dataset $\mathcal{S}^{\mathcal{A}}$, based on which $\mathcal{A}_1$ can construct query tokens $\mathcal{T}^{\mathcal{A}}$. By sending $\mathcal{T}^{\mathcal{A}}$ into the distributed inference framework, $\mathcal{A}_1$ can obtain the corresponding embeddings $\mathcal{E}^{\mathcal{A}}$ from the locally hosted server module. Using the pairs $(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}})$, the adversary can train an attack model to reconstruct the target tokens T^{tgt} from E^{tgt}. Ultimately, decoding T^{tgt} via the client module's tokenizer yields the original prompt S^{tgt}. The main challenge is designing an effective attack model for training on $(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}})$. Specifically, for a query sentence $S^{\mathcal{A}}$, the adversary can obtain a token sequence $T^{\mathcal{A}} = \{t_i\}_{i=1}^{l}$ and its intermediate embeddings $E^{\mathcal{A}} = \{\boldsymbol{e}_i\}_{i=1}^{l}$. Although $|T^{\mathcal{A}}| = |E^{\mathcal{A}}|$, the mapping between these two sets is not one-to-one. In fact, each embedding $\boldsymbol{e}_i$ at position i is conditioned on all tokens before position i, i.e., $\{t_1, \cdots, t_i\}$, as shown in Fig. 2a. Existing sentence reconstruction studies [12, 22, 34, 35, 44] suggest

[1] https://www.reddit.com/r/AskReddit/

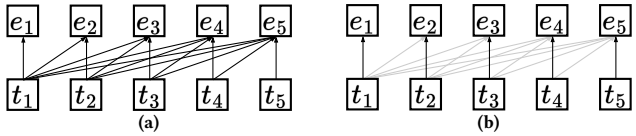


Figure 2: The correspondence between input tokens and intermediate embeddings.

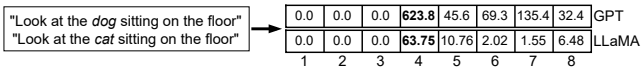
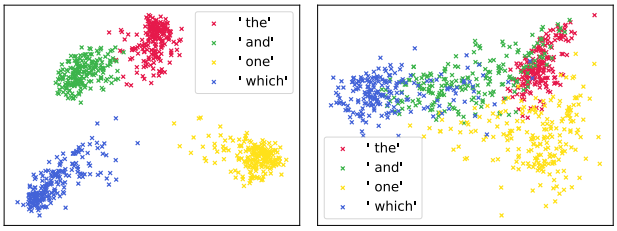


Figure 3: The ℓ_1 norm of token-level embedding differences between S^{dog} and S^{cat} after layer 1 of GPT-2 and Llama-3.2.



(a) After Decoder Layer 6 **(b) After Decoder Layer 16**

Figure 4: The embedding distributions after different layers of Llama-3.2. ' the' denotes that the embedding corresponds to a token sequence ending with the token ' the'.

that transformers, taking $\{\boldsymbol{e}_1, \cdots, \boldsymbol{e}_l\}$ as the input and producing $\{t_1, \cdots, t_l\}$ as the output, are well-suited for this inversion task, as they effectively capture dependencies among sequence elements. However, relying on transformers as the attack model has two significant drawbacks. First, training a robust transformer-based model requires extensive data and computational resources, which may not be feasible in most real-world scenarios. Consequently, transformer-based models are often limited to reconstructing short sequences, e.g., $l \leq 15$ [12]. Second, transformers reconstruct tokens sequentially, so errors in earlier tokens may negatively impact subsequent ones [2]. In this paper, we aim to design more efficient and generalized attacks that eliminate the reliance on transformer-based models, which require new observations on intermediate sequence embeddings of LLMs.

The Key Observation. Our investigation into the correspondence between $T^{\mathcal{A}}$ and $E^{\mathcal{A}}$ reveals a key observation: although $\boldsymbol{e}_i$ is conditioned on $\{t_1, \cdots, t_i\}$, the influence of t_i significantly outweighs that of the preceding tokens $\{t_1, \cdots, t_{i-1}\}$, as demonstrated in Fig. 2b. To illustrate this, consider the example prompts S^{dog} = "Look at the dog sitting on the floor" and S^{cat} = "Look at the cat sitting on the floor". These prompts differ only in their fourth tokens, with t_4^{dog} = ' dog' $\neq$ t_4^{cat} = ' cat', while $t_i^{\text{dog}} = t_i^{\text{cat}}$ for all $i \neq 4$. We first input these two prompts into

[2] To justify this, we first train three transformer-based attack models following [12], whose training datasets and hyperparameters are the same but with different initial parameters. Then, we use a prompt "ash thorp octane photo real organic line 3d glowing" for reconstruction, where the first three tokens are fixed, and the following tokens need to be reconstructed by attack models. The results reconstructed by the three models are ["ash thorp octane *picture Octane: The 3d Story*", 'ash thorp octane *photo 3d photo light*', "ash thorp octane *book is real at the University*"], indicating that the first reconstructed token may influence the following reconstructions, such as picture → Story, and book → University.

an LLM and obtain the corresponding intermediate sequence embeddings after layer 1. After that, we compute the ℓ_1 norm of differences between these two sequence embeddings at each position, $||\boldsymbol{e}_i^{\text{dog}} - \boldsymbol{e}_i^{\text{cat}}||_1$ for all i. Fig. 3 presents the results tested on GPT-2 [42] and Llama-3.2 [31].

The main observation is that the embedding difference at position 4 is substantially larger than at subsequent positions. This large difference at position 4 aligns with the distinct tokens $t_4^{\text{dog}} \neq t_4^{\text{dog}}$. However, the small differences at positions 5 to 8 are counterintuitive, given that each embedding $\boldsymbol{e}_i$ is computed based on all preceding tokens. Intuitively, a significant change in t_4 should propagate to embeddings at positions 5 to 8. Considering that $t_i^{\text{dog}} = t_i^{\text{cat}}$ for $5 \leq i \leq 8$, we conclude that *the influence of the ending token t_i in a sequence dominates the sequence embedding $\boldsymbol{e}_i$*, formalized as:

$$\boldsymbol{e}_i = g(t_i) + \epsilon_g(\{t_1, \cdots, t_{i-1}\}), \tag{1}$$

where g denotes the neural transformation between $\boldsymbol{e}_i$ and $\{t_1, \cdots, t_i\}$, and ϵ_g denotes a perturbation function conditioned on the preceding tokens $\{t_1, \cdots, t_{i-1}\}$. To further validate this observation that $||g(t_i)|| \gg ||\epsilon_g(\{t_1, \cdots, t_{i-1}\})||$, we processed a Wikipedia dataset [30] with Llama-3.2 to obtain embeddings after layers 6 and 16 and map each embedding $\boldsymbol{e}_i$ to its corresponding ending token t_i for each sequence $\{t_1, \cdots, t_i\}$. Using principal component analysis (PCA) [3], we reduced the embedding dimensions to two and visualized the embedding distributions for common tokens $t_i \in \{$' the', ' and', ' one', ' which'$\}$ in Fig. 4. These tokens appear most frequently in English, and their corresponding sequence embeddings can comprehensively capture the influence of preceding tokens on the embeddings of ending tokens. It is worth noting that different points in Fig. 4 correspond to different positions i, i.e., $\boldsymbol{e}_i$ is only mapped to the ending token t_i of a sequence, while the sequence length i can be arbitrary. From Fig. 4, we observe that embeddings $\boldsymbol{e}_i$ in the embedding space form distinct clusters primarily determined by the ending token t_i, while the influence of preceding tokens $\{t_1, \cdots, t_{i-1}\}$ results in minimal intra-cluster shifts. These shifts are significantly smaller than the inter-cluster distances caused by differences in the ending tokens t_i.

The Attack Method. Building on the observations from Fig. 3 and 4, an adversary can first leverage the auxiliary dataset $\mathcal{S}^{\mathcal{A}}$ to obtain $(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}})$, then construct mappings $(\boldsymbol{e}_i, t_i)$ from $(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}})$ and train a classification model $\phi : \boldsymbol{e}_i \rightarrow t_i$ to learn the correspondence between sequence embedding $\boldsymbol{e}_i$ and their ending tokens t_i. Once ϕ is trained, the adversary can use it to reconstruct the target sequence S^{tgt} from E^{tgt} on a token-by-token basis, on the condition that S^{tgt} and $\mathcal{S}^{\mathcal{A}}$ are drawn from similar distributions. Algorithm 1 presents detailed steps for $\mathcal{A}_1$. Since $\mathcal{A}_1$ has unlimited query budgets, collecting a larger $\mathcal{S}^{\mathcal{A}}$ will result in better performance of ϕ. However, our experiments indicate that having 200 occurrences of each token in $\mathcal{S}^{\mathcal{A}}$ suffices to train a high-performing attack model.

Unlike transformer-based methods [12, 22, 34, 35, 44], our approach does not rely on preceding tokens $\{t_1, \cdots, t_{i-1}\}$ for reconstructing t_i, making its reconstruction accuracy independent of sequence length or errors in earlier tokens. Notably, Algorithm 1 can achieve accuracies higher than 90% across all layers in state-of-the-art LLMs, as demonstrated in Section 5.1.

[3] https://en.wikipedia.org/wiki/Principal_component_analysis

Algorithm 1: Adversary $\mathcal{A}_1$ with Unlimited Query Budgets and an Auxiliary Dataset $\mathcal{S}^{\mathcal{A}}$ with $P_{\mathcal{A}} \approx P_{\text{tgt}}$

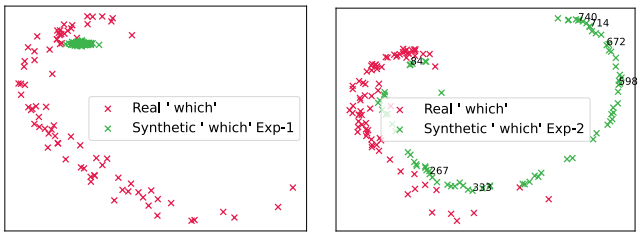
Input: Auxiliary dataset $\mathcal{S}^{\mathcal{A}}$, learning rate α, target intermediate embeddings E^{tgt}
Output: Reconstructed private prompt S^{tgt}

$(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}}) \leftarrow$ Send queries from $\mathcal{S}^{\mathcal{A}}$ for LLM inference ;
$\{(\boldsymbol{e}_i, t_i)\}_{i=1}^{|\mathcal{T}^{\mathcal{A}}|} \leftarrow$ Construct training data from $(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}})$;
$\boldsymbol{\theta}_\phi \leftarrow \mathcal{N}(0, 1)$; `// Initialize the attack model`
foreach *epoch* **do**
  foreach *batch* **do**
    $loss \leftarrow 0$;
    Randomly select a batch of samples ;
    foreach *sample i* **do**
      $\hat{t}_i \leftarrow \phi(\boldsymbol{e}_i; \boldsymbol{\theta}_\phi)$;
      $loss \leftarrow loss + L(\hat{t}_i, t_i)$;
    $\boldsymbol{\theta}_\phi \leftarrow \boldsymbol{\theta}_\phi - \alpha \cdot \nabla_{\boldsymbol{\theta}_\phi} loss$; `// Update the attack model`
$S^{\text{tgt}} \leftarrow$ "" ;
foreach $\boldsymbol{e}_i \in E^{tgt}$ **do**
  $\hat{t}_i \leftarrow \phi(\boldsymbol{e}_i; \theta_\phi)$;
  Decode $\hat{t}_i$ via the tokenizer and append the result to S^{tgt} ;
return S^{tgt};

4.2 The Second Adversary $\mathcal{A}_2$

Algorithm 1 performs effectively under the assumption that S^{tgt} and $\mathcal{S}^{\mathcal{A}}$ follow a similar distribution. However, in practical scenarios, the adversary may lack knowledge of the distribution of S^{tgt}. Consequently, a model ϕ trained on $\mathcal{S}^{\mathcal{A}}$ may not generalize well to S^{tgt}. For example, for embeddings output by layer 32 of GPT-2 trained on a Wikipedia dataset [30], Algorithm 1 can achieve a reconstruction accuracy of 98% on Wikipedia-related prompts but experience a significant drop to 61% for prompts sampled from a Q&A dataset SQuAD2.0 [43]. This degradation occurs because approximately 25% tokens in SQuAD2.0, denoted as $\mathcal{T}^c$, are not covered by the training set, i.e., $\mathcal{T}^{\mathcal{A}} \cap \mathcal{T}^c = \emptyset$. To address this issue, it is necessary to generate embeddings $\hat{\mathcal{E}}^c$ corresponding to $\mathcal{T}^c$ and augment the training dataset $(\mathcal{E}^{\mathcal{A}}, \mathcal{T}^{\mathcal{A}})$ with the $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$ pairs. In the case that $\mathcal{T}^c$ is unknown, we can simply use its superset $\mathcal{T}^c = T_{\text{all}} \setminus \mathcal{T}^{\mathcal{A}}$ in the algorithm design. Let $\mathcal{E}^c$ represent the unknown embeddings in the real context corresponding to $\mathcal{T}^c$. To make the attack model trained on the synthetic embeddings $\hat{\mathcal{E}}^c$ generalize well in real context, the main challenge is to generate $\hat{\mathcal{E}}^c$ with a distribution similar to $\mathcal{E}^c$, specifically in how to generate valid sequences $t_1, \cdots, t_i$ for $t_i \in \mathcal{T}^c$ and $i \in \mathbb{N}^+$.

The Key Observation. Since $\mathcal{T}^c$ is absent from the auxiliary dataset $\mathcal{S}^{\mathcal{A}}$, a straightforward approach to generating real sequences $t_1, \cdots, t_i$ for $t_i \in \mathcal{T}^c$ involves searching the LLM's next-token outputs for $t_i \in \mathcal{T}^c$ when inputting $\mathcal{S}^{\mathcal{A}}$ as the prompts. However, this approach depends on LLM behaviors and does not allow precise control over the appearance of the target tokens t_i. Alternatively, one might use GPT-4o [37] to synthesize token sequences with a prompt like, "Please generate a 10-token sentence with 'test' as the third token". However, GPT-4o performs poorly in this case for two reasons: first, the target token 'test' may be retokenized by GPT-4o's tokenizer into multiple tokens (e.g., 't' and



(a) Fixed Synthetic Position $i = 100$ (b) Fixed Synthetic Sequence

Figure 5: The real and synthetic embedding distributions for the token ' which' evaluated on layer 1 of GPT-2.

'est'), causing GPT-4o to treat the target token as separate units; second, GPT-4o lacks an understanding of token composition in sentences and always output sentences with wrong token positions, such as "He studied hard for the upcoming test". Therefore, instead of synthesizing valid sequences $t_1, \cdots, t_i$ for $t_i \in \mathcal{T}^c$, we focus on directly generating the distribution of $\boldsymbol{e}_i$ as defined in Eq. 1. Notice that the generation of $\boldsymbol{e}_i$ for t_i depends on two factors: the position i of t_i and the preceding tokens $t_1, \cdots, t_{i-1}$. To assess the impact of these two factors on the distribution of $\boldsymbol{e}_i$, we design two experiments. In the first, we fix $i = 100$ and generate 200 sequences by replacing $t_1, \ldots, t_{99}$ with random real sequences sampled from $\mathcal{S}^{\mathcal{A}}$ and t_{100} with a target token ' which'. In the second, we fix a sequence $t_1, \ldots, t_l$ with $l = 800$ and generate 200 new sequences based on it by replacing the token at random positions i with the target token ' which'. We visualize the resulting distributions of $\boldsymbol{e}_i$ after PCA reduction in Fig. 5, where red points represent real-context embeddings sampled from SQuAD2.0, i.e., $\mathcal{E}^c$; and green points represent synthetic distributions, i.e., $\hat{\mathcal{E}}^c$. Position labels i for the green points are provided in Fig. 5b for reference. From Fig. 5, we observe that for a target token t_i, the preceding tokens $t_1, \cdots, t_{i-1}$ control the variance of its embedding $\boldsymbol{e}_i$ along certain dimensions, which are not visible in the figure, while the position i influences the overall shape of $\boldsymbol{e}_i$ along the principal components. Thus, to effectively imitate the distribution of $\mathcal{E}^c$, we need to perturb the embedding of target tokens along both the principal and minor components.

The Attack Method. Building on the insights from Fig. 5, we propose to synthesize the embedding distribution of t_i by constructing δ random sequences $\{t_1, \cdots, t_i\}$ that terminate with the target token $t_i \in \mathcal{T}^c$, where the position i and preceding tokens $\{t_1, \cdots, t_{i-1}\} \subset \mathcal{T}^{\mathcal{A}}$ are randomized, and δ denotes the augmentation factor. The detailed synthesis procedure is outlined in Algorithm 2. This randomized generation approach ensures that the synthetic embeddings can effectively cover the distribution of real sequence embeddings corresponding to t_i, as illustrated in Fig. 12 in the experiments.

Note that this synthesis method also functions as a data augmentation strategy for training the classifier ϕ. By exposing ϕ to the perturbed synthetic embedding distribution $\hat{\mathcal{E}}^c$, which spans a broader range than the real embedding distribution $\mathcal{E}^c$, the robustness and generalization of ϕ are significantly enhanced for reconstructing S^{tgt}. In our experiments, $\mathcal{A}_2$ can considerably improve the reconstruction accuracies from 50% to higher than 90%, closely matching the performance of $\mathcal{A}_1$ under the ideal condition where $P_{\mathcal{A}} \approx P_{\text{tgt}}$.

Algorithm 2: Adversary $\mathcal{A}_2$ with Unlimited Query Budgets and an Auxiliary Dataset $\mathcal{S}^{\mathcal{A}}$ with $P_{\mathcal{A}} \not\approx P_{\text{tgt}}$

Input: Auxiliary dataset $\mathcal{S}^{\mathcal{A}}$, token augmentation factor δ, target intermediate embeddings E^{tgt}
Output: Reconstructed private prompt S^{tgt}

$\mathcal{T}^{\mathcal{A}} \leftarrow$ Obtain the token sets of $\mathcal{S}^{\mathcal{A}}$ via the tokenizer;
$\mathcal{T}^c \leftarrow T_{\text{all}} \setminus \mathcal{T}^{\mathcal{A}}$; `// The token set not in` $\mathcal{S}^{\mathcal{A}}$
$\mathcal{S}^c \leftarrow \emptyset$;
foreach $t^c \in \mathcal{T}^c$ **do**
  foreach $j \in \{1, \cdots, \delta\}$ **do**
    $\{t_1, \cdots, t_n\} \leftarrow$ Sampling a token sequence from $\mathcal{T}^{\mathcal{A}}$;
    $i \leftarrow$ Sampling a position from $\{1, \cdots, n\}$;
    $\mathcal{S}^c \leftarrow \mathcal{S}^c \cup \{t_1, \cdots, t_{i-1}, t^c\}$;
$S^{\text{tgt}} \leftarrow$ Execute Algorithm 1 with inputs $\mathcal{S}^{\mathcal{A}} \cup \mathcal{S}^c$ and E^{tgt};
return S^{tgt};

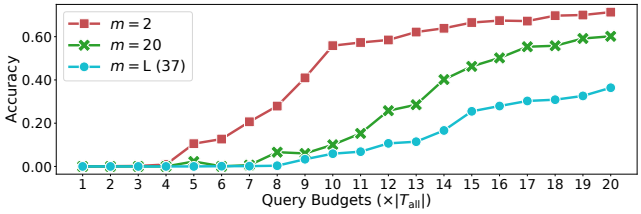


Figure 6: The reconstruction accuracy of $\mathcal{A}_1$ with limited query budgets performed on GPT-2 and WikiText-2 [30].

4.3 The Third Adversary $\mathcal{A}_3$

Both $\mathcal{A}_1$ and $\mathcal{A}_2$ rely on extensive queries to distributed LLM inference frameworks for constructing attack training datasets. However, such queries may impose significant time costs, especially under unstable communication in distributed frameworks, and may incur financial costs when pay-as-you-go API services are used. Therefore, developing a more efficient attack strategy that can work under limited query budgets would be necessary in these scenarios. For the third adversary $\mathcal{A}_3$, apart from the limitation on query budgets, we further remove the auxiliary data assumption, i.e., the attacker has no auxiliary data $\mathcal{S}^{\mathcal{A}}$ for query construction, making this setting the most restrictive one. Here, we follow the leading LLM service providers (e.g., OpenAI [37]) and quantify the query budgets as the maximum number of tokens allowed in total queries.

Challenge. Without any auxiliary dataset $\mathcal{S}^{\mathcal{A}}$, we can still generate the training data $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$ by setting $\mathcal{T}^{\mathcal{A}} = \emptyset$, $\mathcal{T}^c = T_{\text{all}}$ (the set of all unique token IDs), and replacing preceding tokens $\{t_1, \cdots, t_{i-1}\}$ with random token ID combinations. The main problem is about the augmentation factor δ per token ID, i.e., the number of embeddings an attacker can obtain for the same token. Under a limited query budget q_b, δ is constrained to $\lfloor q_b / |T_{\text{all}}| \rfloor$, which may reduce to one, allowing only a single query per token ID. Note that we consider the minimal query budget q_b to be $|T_{\text{all}}|$, as it represents a sufficiently small budget based on the estimates in Table 8, and setting $q_b < |T_{\text{all}}|$ not only restricts queries to a subset of token IDs but also significantly increases the false positive rates for these tokens during reconstruction. The minimal budget $q_b = |T_{\text{all}}|$ can produce a small training dataset, leading to severely underfitted classifiers with poor generalization. For example, Fig. 6 presents

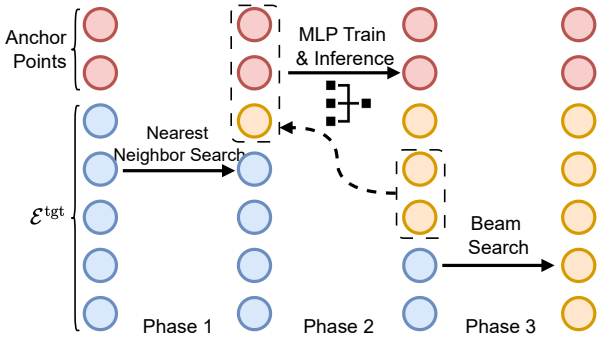


Figure 7: Overview of the semi-supervised three-phase reconstruction framework for $\mathcal{A}_3$. Red points denote the embeddings with labels, i.e., $\hat{\mathcal{E}}^c$; yellow points denote the target embeddings that have been assigned labels; blue points denote the target embeddings to be assigned labels.

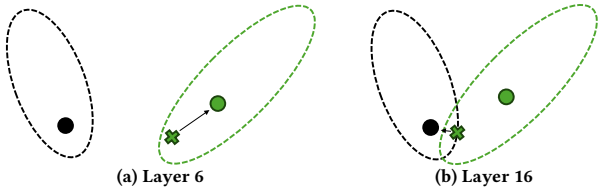


Figure 8: Examples of the nearest neighbor search method. Solid circles denote the anchor points, and the cross denotes the target embedding to be assigned labels.

the testing accuracies of $\mathcal{A}_1$ under limited query budgets, where m denotes the layer number. The results indicate that training a classifier directly with limited query budgets (e.g., $q_b < 10 \times |T_{\text{all}}|$) yields poor performance, particularly for later layers. Therefore, new attack methods need to be designed for this restrictive setting.

Overview. For $\mathcal{A}_3$, we propose a semi-supervised three-phase reconstruction framework. The first two phases involve classification tasks to reconstruct the backbone of sequences with high confidence, while the third phase focuses on semantic reconstruction to fill in unresolved details. Before initializing this attack framework, the adversary needs to construct a sequential token set $\mathcal{T}^c$ with $|\mathcal{T}^c| = q_b$ in a way that the positions and preceding tokens for each token ID are randomized, as inspired by observations from $\mathcal{A}_2$. To put it simply, $\mathcal{T}^c$ can be built by first repeating all tokens in T_{all} at least $\lfloor q_b/|T_{\text{all}}| \rfloor$ times and then randomly shuffling them. After that, the adversary queries the LLM using $\mathcal{T}^c$ and obtains $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$ pairs, where $\hat{\mathcal{E}}^c$ are called the *anchor points*. We will compare the effectiveness of this data generation method with the method used by $\mathcal{A}_2$ in Section 5.3. For effective attacks, q_b *must exceed* $|T_{all}|$ to ensure at least one anchor point per token ID. After obtaining anchor points, the adversary can collect a set of target embeddings $\mathcal{E}^{\text{tgt}}$ and employ the semi-supervised framework to reconstruct $\mathcal{T}^{\text{tgt}}$ and $\mathcal{S}^{\text{tgt}}$. The details of this attack framework are discussed in subsequent sections, with an overview provided in Fig. 7.

4.3.1 Phase 1: Nearest Neighbor Search. As shown in Fig. 4, sequence embeddings with the same ending token typically form a dense cluster in the embedding space, and different clusters are sparsely distributed. Leveraging this property, a straightforward method to assign a token label to a target embedding $\boldsymbol{e}^{\text{tgt}}$ is to compute the average distance from $\boldsymbol{e}^{\text{tgt}}$ to the anchor points in each cluster and assign the token label of the nearest cluster to $\boldsymbol{e}^{\text{tgt}}$:

$$t^{\text{tgt}} = \arg\min_j d^{\text{tgt}}_{\#j}, \text{ where } d^{\text{tgt}}_{\#j} = \sum_{\boldsymbol{e}^c_{\#j} \in E^c_{\#j}} ||\boldsymbol{e}^{\text{tgt}} - \boldsymbol{e}^c_{\#j}||_p / |E^c_{\#j}|, \quad (2)$$

where $\#j$ denotes the token ID j associated with a cluster, and $E^c_{\#j} \subset \hat{\mathcal{E}}^c$ denotes all anchor points in this cluster.

However, this method assumes that all embedding clusters exhibit isotropic distributions, which is generally invalid for real-world datasets, as demonstrated by the red points in Fig. 5. As a result, it can easily lead to errors when clusters overlap, particularly in later layers of LLMs. For example, in Fig. 8b, the target embedding, represented by a cross, is mistakenly assigned to the black cluster by nearest neighbor search, although its true label is green. Additionally, with a limited number of anchor points, it is challenging to accurately capture the overall shapes of embedding clusters, further reducing prediction accuracy. To mitigate this issue, we propose incorporating a confidence score for the predictions generated by the nearest neighbor search. This allows us to focus on assigning labels only to target embeddings with high confidence. The confidence score is calculated as follows. For a target embedding $\boldsymbol{e}^{\text{tgt}}$, we first compute the k smallest cluster distances $\{d^{\text{tgt}}_{\#j_1}, \cdots, d^{\text{tgt}}_{\#j_k}\}$. Then, for each $o \in \{j_1, \cdots, j_k\}$,

$$\hat{d}^{\text{tgt}}_{\#o} = (c_1 - d^{\text{tgt}}_{\#o})/c_2, \quad (3)$$

$$p^{\text{tgt}}_{\#o} = \frac{\exp(\hat{d}^{\text{tgt}}_{\#o})}{\sum_{r=j_1}^{j_k} \exp(\hat{d}^{\text{tgt}}_{\#r})}, \quad (4)$$

where Eq. 3 normalizes the distances with constants c_1 and c_2, and Eq. 4 applies softmax to the normalized distances. Next, we compute the entropy of the normalized probabilities:

$$H(t^{\text{tgt}}) = -\sum_{o=j_1}^{j_k} p^{\text{tgt}}_{\#o} \log p^{\text{tgt}}_{\#o}, \quad (5)$$

The intuition behind Eq. 5 is that if $\boldsymbol{e}^{\text{tgt}}$ is significantly closer to a cluster $j \in \{j_1, \cdots, j_k\}$ than to other $k-1$ clusters, the probability $p^{\text{tgt}}_{\#j}$ of $\boldsymbol{e}^{\text{tgt}}$ belonging to cluster j will be higher, resulting in lower entropy $H(t^{\text{tgt}})$, i.e., less uncertainty for the results $t^{\text{tgt}} = j$. The confidence score for the result t^{tgt} from Eq. 2 is then computed as $1 - H(t^{\text{tgt}})$. Specifically, if $H(t^{\text{tgt}})$ is below a small threshold τ_{NN} (the confidence is larger than $1 - \tau_{\text{NN}}$), we assign t^{tgt} to $\boldsymbol{e}^{\text{tgt}}$ and incorporate $(\boldsymbol{e}^{\text{tgt}}, t^{\text{tgt}})$ into the anchor points. After processing all target embeddings in $\mathcal{E}^{\text{tgt}}$, the method proceeds to the next phase.

4.3.2 Phase 2: Classifier Search. The nearest neighbor search, limited by its assumption of isotropic cluster distributions, can only assign labels to a small fraction of $\mathcal{E}^{\text{tgt}}$, primarily when clusters are sparsely distributed. However, by augmenting the anchor points $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$ during this process, we obtain additional samples for each cluster, which can enable the training of a non-linear classifier ϕ akin to the approach used in $\mathcal{A}_1$. This classifier can learn the anisotropic distributions of different clusters more effectively. It is important to note that the sample space $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$ may remain small under restricted query budgets after the first phase, thus applying a large gradient penalty during classifier training is essential to mitigate overfitting issues. Once ϕ is trained, it can be employed

to predict labels for target embeddings that remain unlabeled after the nearest neighbor search.

Despite this, ϕ's accuracy is likely constrained by the insufficient representation of real embedding distributions within $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$. To enhance prediction reliability, we adopt a similar approach to Eq. 5, computing logits entropy from the softmax outputs of ϕ. Labels are assigned only to embeddings with logits entropy below a predefined threshold τ_ϕ. This model training and inference phase can iterate multiple times to progressively expand the anchor points for training different ϕs. For any embeddings that remain unlabeled after this phase, the framework proceeds to the third phase.

4.3.3 Phase 3: Beam Search. Following the first two phases, partially reconstructed sentences may resemble: "The[...] is a pioneer in[...] programming aimed at minorities[...] and[...]", where "[...]" denotes a token failed to be reconstructed. In this phase, we design a modified beam search to fill in the missing tokens. Beam search is commonly used for generating plausible sentences in language models [14], which operates by generating tokens sequentially while retaining only the β (beam width) most probable sequences at each step. The selection of the most probable sequences is guided by probabilities output by a language model. For this task, we propose to use a shadow LLM f_{shadow} to fulfill the beam search, which can be any pre-trained open-source LLM, such as GPT-2. Considering that our task is a token reconstruction task instead of a sentence generation task, we modify the traditional beam search method [14] to a context-aware and candidate-constrained search method. Algorithm 3 presents details of the modified beam search method.

Specifically, the "context-aware" modification ensures that we determine the probabilities of possible missing tokens based on the reconstructed context, i.e., tokens already assigned labels (line 7). Only those missing tokens that maximize the output probabilities of f_{shadow} for the given context are retained (line 11). This modification leverages the semantic knowledge embedded in f_{shadow}, ensuring that the reconstructed tokens are semantically consistent with the given context. The "candidate-constrained" modification restricts the search space for each missing token to the top-5 candidates identified during the nearest neighbor search (line 6), rather than exploring the entire token dictionary. Table 2 presents the top-k ($k \in \{1, 5, 10\}$) accuracies obtained by the nearest neighbor search under the minimum query budget, from which we observe that the top-5 accuracies are typically 30% higher than the top-1 accuracies, while accuracy gains from top-5 to top-10 are minimal. Consequently, we focus on reconstructing the missing tokens from the top-5 candidates via beam search, rather than generating sentences solely for plausible semantics. In the final reconstructed sentences, tokens recovered through beam search can be flagged to highlight potential errors, thereby reducing the risk of transmitting misleading information to the adversary. We summarize the semi-supervised framework of $\mathcal{A}_3$ in Algorithm 4.

Algorithm 3: Context-Aware Constrained Beam Search

Input: Beam width β, language model f_{shadow}, partially reconstructed token sequence T^{tgt}, mask for missing tokens M_{miss}, top-5 candidate tokens C_{t5}
Output: Best sequence S^*

1 Beam $\leftarrow \{\langle \texttt{start}, 0\rangle\}$; `// Sequence and its score`
2 **for** $i = 1$ ***to*** $|T^{tgt}|$ **do**
3 Cands $\leftarrow \emptyset$; `// Candidate sequences`
4 **foreach** $\langle S, score\rangle \in Beam$ **do**
5 **if** $M_{miss}[i]$ *is true* **then**
6 **foreach** $w \in C_{t5}[i]$ **do**
7 new_score $\leftarrow$ score $+ \log f_{\text{shadow}}(w \mid S)$;
8 Cands $\leftarrow$ Cands $\cup \langle S \| w, \text{new_score}\rangle$;
9 **else**
10 Cands $\leftarrow$ Cands $\cup \langle S \| T^{\text{tgt}}[i], \text{score}\rangle$;
11 Beam $\leftarrow$ Top-β(Cands) ;
12 $S^* \leftarrow \arg\max_{\langle S, \text{score}\rangle \in \text{Beam}}$ score ; `// Best sequence`
13 **return** S^*;

Table 2: The accuracies for ℓ_1-norm nearest neighbor search, tested on GPT-2 and Wikitext with $q_b = |T_{\text{all}}|$.

Accuracy	Layers			
	2	10	20	30
Top-1	0.7747	0.4480	0.2882	0.1149
Top-5	0.8802	0.5689	0.3958	0.1800
Top-10	0.8984	0.5930	0.4387	0.2108

5 Experiments

Platform. All attack algorithms are implemented in Python using *PyTorch* [4]. Experiments are conducted on a high-performance server equipped with AMD EPYC 7742 64-Core Processor × 256, NVIDIA A100-SXM4-40GB × 4, and 256GB RAM, running Ubuntu 22.04. We follow the settings of Petals [4, 5] to set up a local distributed LLM inference framework [5] and perform experiments on it.

LLMs. We evaluate the proposed attacks on three state-of-the-art decoder-only LLMs: Phi-3.5 [32], Llama-3.2 [31], and GPT-2 [6], which are mainly designed for next-token prediction tasks. Additionally, we assess the attacks on BERT [8], an encoder-only model primarily used for sentence embedding generation. The key architectural distinction is that decoder-only models condition the generation of embeddings $\boldsymbol{e}_i$ on the preceding $t-1$ tokens, as illustrated in Fig. 2a, whereas encoder-only models generate embeddings $\boldsymbol{e}_i$ based on both preceding and following tokens within the input sequence. By including BERT in the evaluation, we aim to test the generalizability of the proposed attacks across different types of LLM architectures. The LLM details are shown in Table 3.

Datasets. We utilize four datasets in our experiments: WikiText-2 [30], SQuAD 2.0 [43], Midjourney prompts [2], and PrivatePrompts [35]. *WikiText-2* comprises over 100 million tokens extracted from high-quality and featured Wikipedia articles. *SQuAD 2.0* includes over 150,000 crowd-sourced questions and is designed for evaluating the reading comprehension ability of LLMs. *Midjourney prompts* include 250,000 text-to-image messages collected from the Midjourney bot server over four weeks. *PrivatePrompts* consists of 251,270 manually-crafted sensitive strings with inserted PII entities, such as name and date. For $\mathcal{A}_1$, each dataset is randomly partitioned into training, testing, and evaluation splits in the ratio

[4] https://pytorch.org/

[5] https://github.com/bigscience-workshop/petals

Algorithm 4: Adversary $\mathcal{A}_3$ with Limited Budgets

Input: Budgets q_b, distance entropy threshold τ_{NN}, logits entropy threshold τ_ϕ, beam width β, target embeddings $\mathcal{E}^{\text{tgt}}$
Output: Reconstructed private prompt set $\mathcal{S}^{\text{tgt}}$

$\delta \leftarrow \lfloor q_b / |T_{\text{all}}| \rfloor$;
$\mathcal{T}^c \leftarrow$ Randomly repeat and combine all $t \in T_{\text{all}}$ to form a q_b-token sequence ;
if $\delta > 100$ **then**
  $\mathcal{S}^{\text{tgt}} \leftarrow$ Execute Algorithm 1 with inputs $\mathcal{T}^c$ and $\mathcal{E}^{\text{tgt}}$;
  return $\mathcal{S}^{\text{tgt}}$;
$\hat{\mathcal{E}}^c \leftarrow$ Send queries from $\mathcal{T}^c$ for distributed LLM inference ;
$\{\langle E^c_{\#j}, \#j \rangle\}_{j=1}^{|T_{\text{all}}|} \leftarrow$ Construct anchor points from $(\hat{\mathcal{E}}^c, \mathcal{T}^c)$;
$\mathcal{S}^{\text{tgt}} \leftarrow \varnothing$;
$C_{\text{top-}k} \leftarrow \varnothing$;
foreach $\boldsymbol{e}_i^{tgt} \in \mathcal{E}^{tgt}$ **do** `// Phase 1: nearest neighbor search`
  $\{t^{\text{tgt}}_{i\#j_1}, \cdots, t^{\text{tgt}}_{i\#j_k}\} \leftarrow$ Compute top-k candidates;
  $C_{\text{top-}k} \leftarrow C_{\text{top-}k} \cup \{t^{\text{tgt}}_{i\#j_1}, \cdots, t^{\text{tgt}}_{i\#j_k}\}$;
  $H_{\text{NN}} \leftarrow$ Compute distance entropy for $t^{\text{tgt}}_{i\#j}$ using Eq. 2 and 5;
  if $H_{NN} < \tau_{NN}$ **then**
    $\mathcal{S}^{\text{tgt}} \leftarrow \mathcal{S}^{\text{tgt}} \cup \langle t^{\text{tgt}}_{i\#j}, i \rangle$;
    $E^c_{\#j} \leftarrow E^c_{\#j} \cup \boldsymbol{e}_i^{\text{tgt}}$;
foreach *iteration* **do** `// Phase 2: classifier search`
  $\phi \leftarrow$ Train a classifier on anchor points;
  foreach $\boldsymbol{e}_i^{tgt} \in \mathcal{E}^{tgt} - \textit{anchor points}$ **do**
    $\{p^{\text{tgt}}_{i\#j}\}_{j=0}^{|T_{\text{all}}|-1} \leftarrow \phi(\boldsymbol{e}_i^{\text{tgt}})$;
    $H_\phi \leftarrow$ Compute logits entropy;
    if $H_\phi < \tau_\phi$ **then**
      $\mathcal{S}^{\text{tgt}} \leftarrow \mathcal{S}^{\text{tgt}} \cup \langle \arg\max_j p^{\text{tgt}}_{i\#j}, i \rangle$;
      $E^c_{\#j} \leftarrow E^c_{\#j} \cup \boldsymbol{e}_i^{\text{tgt}}$;
if $|\mathcal{S}^{tgt}| < |\mathcal{E}^{tgt}|$ **then** `// Phase 3: beam search`
  $T^{\text{tgt}}, M_{\text{miss}} \leftarrow$ Construct partial sequence and mask from $\mathcal{S}^{\text{tgt}}$;
  $S^* \leftarrow$ Execute Algorithm 3 with inputs $\beta, T^{\text{tgt}}, M_{\text{miss}}$ and $C_{\text{top-}k}$;
  Update $\mathcal{S}^{\text{tgt}}$ based on S^*;
return $\mathcal{S}^{\text{tgt}}$;

Table 3: The used large language models.

LLM	#params	#layers	Release Year	Type	Developer
Phi-3.5	3.82B	32	2024	Decoder-only	Microsoft
Llama-3.2	1.24B	16	2024	Decoder-only	Meta
GPT-2	812M	36	2019	Decoder-only	OpenAI
BERT	335M	24	2018	Encoder-only	Google

(64%, 16%, 20%). The training and testing subsets are used for attack model development, while the evaluation subset is held out for attack evaluation. For $\mathcal{A}_2$, one dataset is used exclusively for training and testing, while another is designated for attack evaluation. For $\mathcal{A}_3$, all datasets are used only for attack evaluation.

Attack Implementation. The classifier ϕ for all three attacks is implemented using a multi-layer perceptron (MLP) with six layers, where layer normalization and ReLU activation are used between consecutive layers. The input dimension matches the embedding size of the target LLM, while the output dimension corresponds to the tokenizer's vocabulary size, $|T_{\text{all}}|$. The MLP is trained for six epochs during $\mathcal{A}_1$ and $\mathcal{A}_2$ and for four epochs per iteration during $\mathcal{A}_3$ of Algorithm 4. The distance and logits thresholds in Algorithm 4 are set to $\tau_{\text{NN}} = \tau_\phi = 0.05$. The beam width β in the beam search phase is set to 6.

Metric. The effectiveness of the proposed attacks is measured by two metrics: token Reconstruction Accuracy (RA) and Cosine Semantic Similarity (CSS). RA is calculated as: $\text{RA} = \sum_{t_i \in \mathcal{T}^{\text{tgt}}} \frac{\mathbb{1}(\hat{t}_i = t_i)}{|\mathcal{T}^{\text{tgt}}|}$, where $\mathbb{1}$ is the indicator function, and $\hat{t}_i$ denotes the reconstructed token corresponding to t_i. CSS is calculated by first extracting semantic embeddings of both reconstructed and ground truth prompts using Sentence-BERT [6] and then measuring the cosine similarity between them. Based on our experiments, RA > 40% is considered sufficient for plausible reconstruction, as key concepts of the ground truth prompts can already be reconstructed under this accuracy.

Baselines. Three baselines are used in our experiments: two sentence embedding inversion attacks designed primarily for BERT-like LLMs, *B-SEI* [12] and *Vec2Text* [34], and one LLM output inversion attack, *Output2Prompt* [49]. B-SEI and Vec2Text can be adapted to reconstruct prompts from different LLM layers, while Output2Prompt can only use LLM outputs for prompt reconstruction. Although Output2Prompt operates under different assumptions than embedding inversion attacks, comparing it with the others helps assess vulnerabilities across the entire LLM inference pipeline.

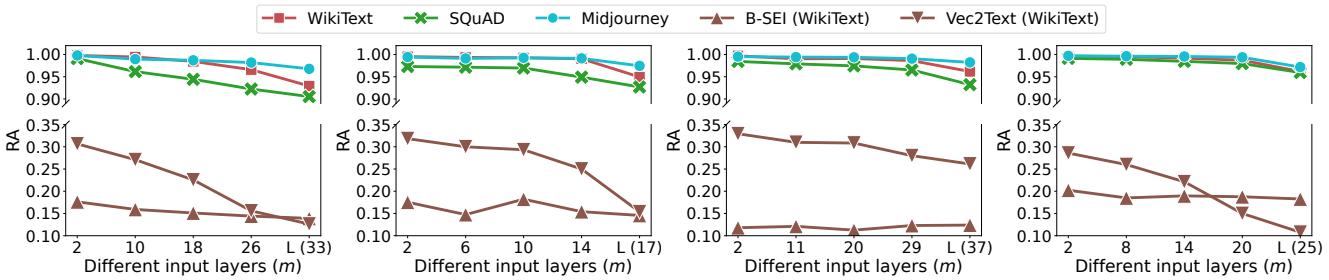
5.1 Attack Performance of $\mathcal{A}_1$

We evaluate the performance of the first attack $\mathcal{A}_1$ across different layers of various LLMs and datasets. Note that in the distributed LLM inference pipeline (Fig. 1), m denotes the first decoder layer hosted by the adversary. An attack on layer m implies reconstructing prompts based on the embeddings $E^{(m-1)}$ fed into layer m. Typically, $m \geq 2$ in our experiments, as $m = 1$ corresponds to reconstructing prompts directly from initial token embeddings, which is trivial due to the one-to-one correspondence between these embeddings and input token IDs. Considering that different LLMs have different numbers of layers (n), for each LLM, we evaluate the attack performance on five evenly selected layers spanning from the first to the last. Note that $m = \text{L}$ (i.e., $n + 1$) implies reconstructing prompts from the embeddings fed into the linear layer.

Results of $\mathcal{A}_1$. The reconstruction accuracies of $\mathcal{A}_1$ are shown in Fig. 9, from which we draw three key observations. *First*, the RA of $\mathcal{A}_1$ decreases as the layer number increases. *Second*, $\mathcal{A}_1$ achieves higher RA on encoder-based BERT compared to decoder-based LLMs. *Third*, $\mathcal{A}_1$ consistently achieves RA greater than 90% across all layers in the evaluated models. To explain these observations, we visualize embedding distributions across different LLM layers by first using PCA to project the embeddings into two dimensions and then scatter these two-dimensional points in Fig. 10.

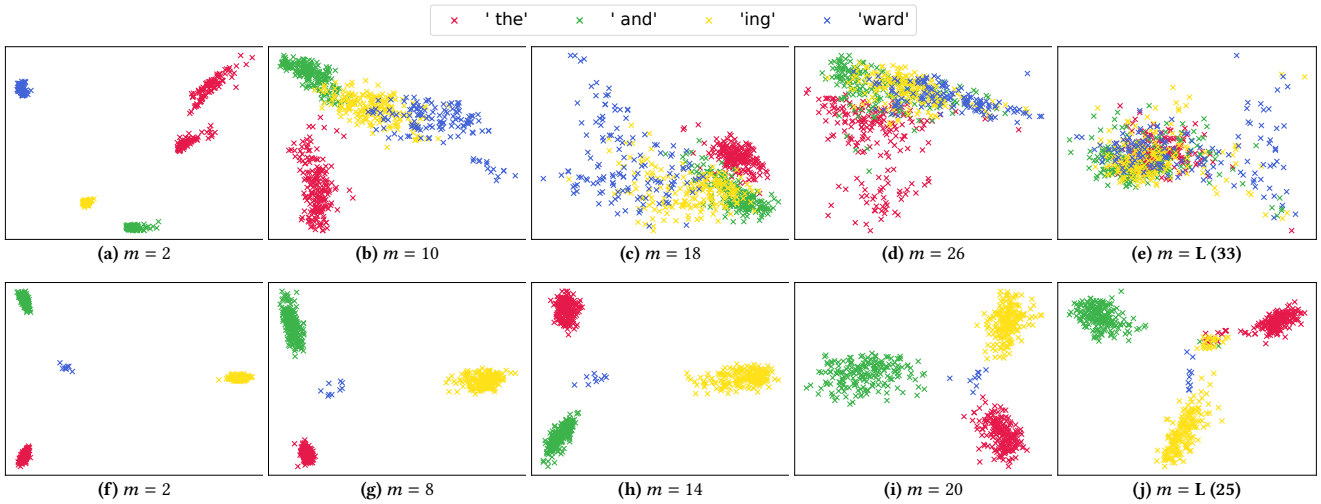
Rationale. From Fig. 10, we can see that embeddings corresponding to different tokens are sparsely distributed in the early layers. As the inference progresses, embedding clusters diffuse and overlap, reducing the separability between clusters and thus hindering the adversary's classifier. This diffusion explains the first observation that RA decreases in deeper layers. Additionally, Fig. 10 reveals that

[6] https://huggingface.co/efederici/sentence-bert-base



(a) Phi-3.5 (b) Llama-3.2 (c) GPT-2 (d) BERT

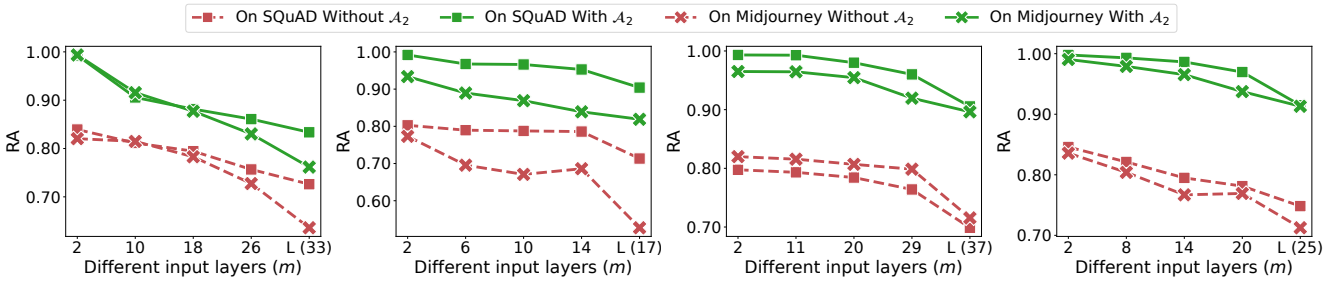
Figure 9: The reconstruction accuracies of $\mathcal{A}_1$ performed on different models and datasets. Reconstruction examples are deferred to the appendix of the full version [27].



(a) $m = 2$ (b) $m = 10$ (c) $m = 18$ (d) $m = 26$ (e) $m = \mathrm{L}$ (33)

(f) $m = 2$ (g) $m = 8$ (h) $m = 14$ (i) $m = 20$ (j) $m = \mathrm{L}$ (25)

Figure 10: The embedding distributions generated in WikiText-2 via (a)–(e) Phi-3.5 and (f)–(j) BERT for $\mathcal{A}_1$. See the appendix of the full version [27] for more results.



(a) Phi-3.5 (b) Llama-3.2 (c) GPT-2 (d) BERT

Figure 11: The reconstruction accuracies of $\mathcal{A}_2$ performed on different models and datasets. The auxiliary dataset is WikiText-2. Reconstruction examples are deferred to the appendix of the full version [27].

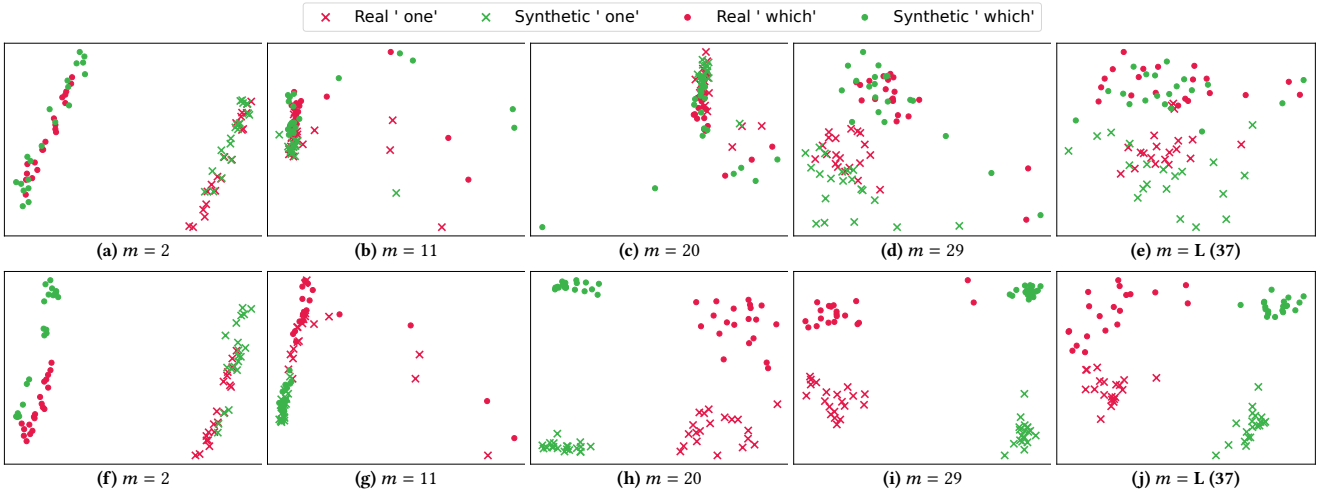
cluster diffusion in encoder-based BERT is less pronounced compared to decoder-based LLMs. This behavior can be attributed to BERT's training strategy, which involves randomly masking input tokens and making BERT predict these masked tokens [8]. This pre-training objective encourages BERT to generate token embedding clusters with better distinguishability, thus creating more benefits for the proposed attacks and explaining our second observation. The third observation relates to the classification features utilized by the attack model. Note that the embedding dimension is typically large in LLMs, such as 3072 for Phi-3.5, while Fig. 10 only visualizes the first two principal components of embeddings. Although the embedding clusters diffuse and overlap in the later layers along the first two components as shown in Fig. 10e, the attack classifier can leverage information from additional dimensions of the embeddings that are not visible in this projection. Consequently, $\mathcal{A}_1$ can maintain over 90% RA even in deeper layers, despite the apparent cluster diffusion in the visualized dimensions. Fig. 9 also shows that our attack significantly outperforms the baseline method. The reason has been discussed in Section 1.

Table 4: Performance comparison between $\mathcal{A}_2$ and the baselines. WikiText-2 and PrivatePrompts are the auxiliary and test datasets, respectively. O denotes LLM output, with BERT outputs generated via a BertGeneration [15] model.

Metric	Method	Phi-3.5				Llama-3.2				GPT-2				BERT			
		2	18	L (33)	O	2	10	L (17)	O	2	20	L (37)	O	2	14	L (25)	O
RA	$\mathcal{A}_2$	**0.9744**	**0.8439**	**0.8074**	/	**0.9212**	**0.8945**	**0.8112**	/	**0.9259**	**0.9144**	**0.8389**	/	**0.9833**	**0.9575**	**0.8598**	/
	B-SEI	0.1862	0.1591	0.1411	/	0.1856	0.1541	0.1375	/	0.1480	0.1219	0.1126	/	0.2023	0.1896	0.1826	/
	Vec2Text	0.3065	0.2255	0.1262	/	0.3183	0.2633	0.1538	/	0.3393	0.3084	0.2913	/	0.2858	0.2013	0.1078	/
	Output2Prompt	/	/	/	0.3895	/	/	/	0.2987	/	/	/	0.2059	/	/	/	0.4091
CSS	$\mathcal{A}_2$	**0.9791**	**0.8953**	**0.8502**	/	**0.9676**	**0.9622**	**0.9382**	/	**0.9475**	**0.9276**	**0.8589**	/	**0.9863**	**0.9753**	**0.9393**	/
	B-SEI	0.4828	0.4435	0.4358	/	0.4710	0.4185	0.4181	/	0.4281	0.4031	0.3766	/	0.4745	0.4759	0.4626	/
	Vec2Text	0.5958	0.5366	0.4364	/	0.6012	0.5980	0.5131	/	0.6197	0.6021	0.6041	/	0.6052	0.4572	0.3864	/
	Output2Prompt	/	/	/	0.6053	/	/	/	0.6153	/	/	/	0.5596	/	/	/	0.5992

Table 5: Reconstructed examples generated by different methods on Llama-3.2, with PII entities highlighted.

Method	Layer	CSS		
			`Is this person male or female? output m for male and f for female. Name: Olena Henricksen`	`Task: Find out what day of the week is it on 07 November 1960. 07 November 1960`
$\mathcal{A}_2$	L (17)	0.9382	`Hasthis person male or female: output m for male and f for female. Name:Olena Henricksen`	`Cast: Find out what day of the week is it on 07 November 1960gy 07 November 1960`
B-SEI	L (17)	0.4181	`The person might be male.`	`Guess the day on which weekday`
Vec2Text	L (17)	0.5131	`Given a person's age, tell me whether he is male or female. Output:`	`Task: Find out the year when it falls on the calendar. Input: January 1, 2020 Output:`
Output2Prompt	Output	0.6153	`What is the probability that males in the population is less than 5? - f henricksen`	`What is the date on 07 November 1960?`



(a) $m = 2$ (b) $m = 11$ (c) $m = 20$ (d) $m = 29$ (e) $m = \mathrm{L}\ (37)$

(f) $m = 2$ (g) $m = 11$ (h) $m = 20$ (i) $m = 29$ (j) $m = \mathrm{L}\ (37)$

Figure 12: Comparison between real and synthetic embedding distributions generated in GPT-2 via (a)–(e) $\mathcal{A}_2$ and (f)–(j) $\mathcal{A}_3$.

5.2 Attack Performance of $\mathcal{A}_2$

In this experiment, we evaluate the performance of $\mathcal{A}_2$ across different models, focusing on the improvements achieved through the data generation method in Algorithm 2. Specifically, we train attack models on WikiText-2 both with and without performing Algorithm 2 and then evaluate their performance on SQuAD 2.0 and Midjourney prompts. In this setup, the token augmentation factor δ in Algorithm 2 is fixed at 256. Note that its impact on attack performance is analogous to the influence of limited query budgets on $\mathcal{A}_1$, which is illustrated in Fig. 6.

Results of $\mathcal{A}_2$. The results are presented in Fig. 11, where green lines represent reconstruction accuracies of $\mathcal{A}_1$ after applying Algorithm 2, and red lines correspond to the direct performance of $\mathcal{A}_1$ across different datasets. From these results, we see that $\mathcal{A}_2$ demonstrates a substantial improvement (typically 20%) in attack performance without prior knowledge of target domains, which is comparable to the performance of $\mathcal{A}_1$ trained and tested under the same domain. Moreover, the decrease in reconstruction accuracy from the first to the last layer is less pronounced for the green lines compared to the red lines, indicating that Algorithm 2 enhances both the robustness and generalization of the attack model.

Rationale. The performance improvement brought by $\mathcal{A}_2$ is closely related to the effectiveness of the embedding generation method in Algorithm 2. To further analyze the effectiveness of Algorithm 2, we visualize real and synthetic embeddings corresponding to two tokens ' one' and ' which' in Fig. 12a–12e, where red denotes real embeddings, green denote synthetic embeddings, and different markers denote different tokens. These visualizations illustrate that embeddings generated by Algorithm 2 (green) effectively simulate the real distributions (red) of different tokens across various layers, validating the performance improvements shown in Fig. 11.

To better understand the relationship between $\mathcal{A}_2$ performance and distributional differences between auxiliary and test datasets, we quantify the dataset distributional differences using Total Variation (TV) distance [7], which ranges from 0 (identical distributions) to 1 (no distribution overlap). The TV between WikiText-2 and SQuAD2.0 is 0.4998, while the TV between WikiText-2 and Midjourney prompts is 0.7319, indicating that SQuAD2.0 shares greater distributional overlap with WikiText-2 compared to Midjourney prompts. This explains why $\mathcal{A}_2$ trained on WikiText-2 consistently performs better on SQuAD2.0 than on Midjourney prompts.

Comparison with Baselines. We compare $\mathcal{A}_2$ with baseline methods using the PII dataset PrivatePrompts, with results shown in Table 4 and reconstructed examples in Table 5. Table 4 indicates that the baselines achieve higher CSS but lower RA, suggesting that they are better at semantic than token-level reconstructions. Table 5 further shows that B-SEI and Vec2Text fail to accurately reconstruct PII entities despite high CSS, indicating that the semantic metric CSS may be inadequate in this context. Although Output2Prompt performs better at reconstructing PII entities, it relies on plaintext LLM outputs rather than embedding vectors, which may be inaccessible in distributed LLM frameworks.

Query Cost. The benefits of $\mathcal{A}_2$ come with significant query costs incurred by Algorithm 2. Table 8 provides a comparison of input token requirements for different attacks and the corresponding estimated costs based on the current pricing policies of Gemini 1.5 Pro and GPT-4o. Notice that $\mathcal{A}_2$ incurs query costs approximately 30 times higher than $\mathcal{A}_1$, which may render it impractical for adversaries with limited query budgets.

5.3 Attack Performance of $\mathcal{A}_3$

Data Generation. Given the constraints of limited query budgets, $\mathcal{A}_3$ employs a random data generation method (lines 2 and 6 in Algorithm 4). To assess its effectiveness, we compare it with the data generation approach in $\mathcal{A}_2$ by visualizing the synthetic embeddings produced by $\mathcal{A}_3$ in Fig. 12f–12j. These visualizations show that the synthetic embeddings (denoted by green) can effectively approximate the real token distributions in early layers. However, as inference progresses, the synthetic embedding distributions gradually deviate from the real ones, which can influence the performance of $\mathcal{A}_3$ in later layers, as discussed below.

Results of $\mathcal{A}_3$. The performance of $\mathcal{A}_3$ on different layers and query budgets are shown in Fig. 13a–13b and Fig. 13c–13d, respectively. The minimum query budget, $|T_{\text{all}}|$, is used for Fig. 13a–13b. Reconstructed examples are provided in Table 6. From Fig. 13a–13b, we observe that even under the minimum budget, $\mathcal{A}_3$ can achieve RA higher than 50% in early layers. But in later layers, the RA decreases to a low level. This observation can be attributed to two factors. First, the sparse embedding distributions in early layers enhance the robustness of the nearest neighbor search. Conversely, in later layers, the clusters diffuse and overlap, which reduces cluster separability and increases errors in the nearest neighbor search process. Meanwhile, the classifier search phase also benefits little from small query budgets, leading to low accuracy gains to $\mathcal{A}_3$. Second, the random data generation method struggles to replicate real distributions accurately in later layers, as shown in Fig. 12j, impacting the accuracy of the nearest neighbor and classifier search phases. Fig. 13c–13d further shows that increasing the query budget improves reconstruction accuracy. The reason is straightforward: larger budgets provide more anchor points for each token cluster, which can better simulate the real cluster distributions and enhance the nearest neighbor search. Table 7 provides more results of $\mathcal{A}_3$ tested on Phi-3.5 and Llama-3.2. In addition, Table 8 demonstrates that $\mathcal{A}_3$ requires significantly fewer query budgets than $\mathcal{A}_1$ and $\mathcal{A}_2$, making it an efficient option under strict budget constraints.

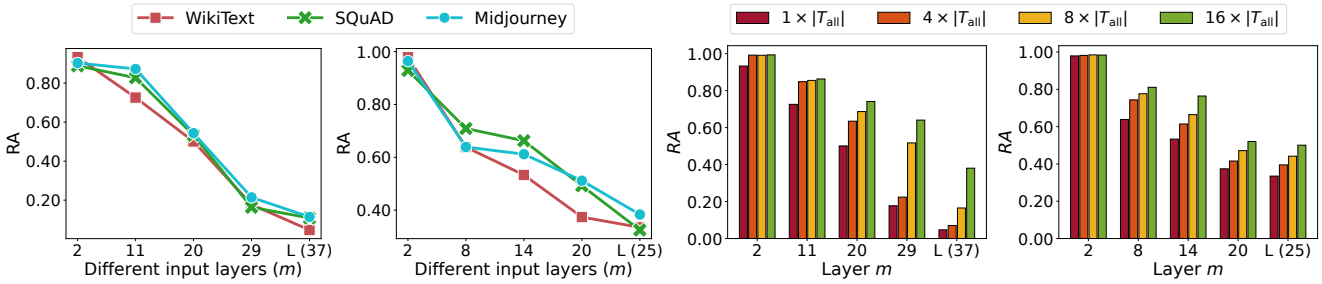
Ablation Study. To assess the contributions of each phase in $\mathcal{A}_3$, we evaluate three branches: nearest neighbor search only (P1), nearest neighbor plus classifier search (P12), and the full three-phase attack (P123). For ease of comparison, we fix the query budgets to $8 \times |T_{\text{all}}|$ and perform attacks on GPT-2 across different datasets. The results are summarized in Table 9.

From branch P1, we observe that the nearest neighbor search performs well in early layers ($\approx$ 100% accuracy at layer 2) but poorly in later layers ($<$ 10% at the last layer). The reason is that the sparsity of embedding clusters can greatly impact the accuracy of the nearest neighbor search, as discussed before. Branches P12 demonstrate that adding classifier search to P1 shows limited gains in early layers, where prompt reconstruction is already nearly complete after nearest neighbor search. However, in later layers, classifier search significantly boosts attack performance by using nonlinear decision boundaries to separate overlapping clusters. Branch P123 shows that the beam search phase provides the most significant improvement in reconstruction accuracy at layer 29. Recall that the beam search is a complementary phase to the first two phases, whose effectiveness depends on the accurate reconstruction of key concepts during earlier phases. When the accuracy of the initial phases is low, the performance enhancement provided by beam search becomes limited. Nevertheless, it is important to note that beam search can recover tokens that, while not identical to the originals, often retain equivalent semantics, for example, ' the'→' The', ' of'→' for', and ' 10'→' ten'. This capability enhances the overall interpretability and coherence of the reconstructed prompts. Additional examples illustrating this phenomenon are provided in the appendix of the full version [27].

6 Possible Countermeasures

Full Pipeline Encryption. Previous studies [3, 23, 33] have proposed to encrypt entire collaborative learning frameworks using holomorphic encryption or secure multi-party computation protocols [20]. Considering that these encryption protocols only support basic computations, such as additions and multiplications, while the activation functions in neural networks are non-linear, these studies [3, 23, 33] often rely on low-degree polynomials to approximate non-linear computations. This approach ensures that all intermediate results exchanged between participants are encrypted, thereby mitigating privacy risks associated with plaintext intermediate data, as explored in this paper. However, a key drawback of this approach is the substantial communication and computation overhead introduced by encryption protocols, which significantly degrades the efficiency of LLM inference. For example, using the method in [23] to predict a single token on Llama-7B takes 14 minutes —

[7] https://en.wikipedia.org/wiki/Total_variation_distance_of_probability_measures



(a) GPT-2 (b) BERT (c) GPT-2 (d) BERT

Figure 13: The reconstruction accuracies of $\mathcal{A}_3$ *w.r.t.* (a)–(b) different layers and (c)–(d) different query budgets. The query budgets in (a)–(b) are $1 \times |T_{\text{all}}|$. The test dataset in (c)–(d) is WikiText-2. More results are in the appendix of the full version [27].

Table 6: Reconstructed examples in $\mathcal{A}_3$. GPT-2 is used for testing. The prompt is selected from SQuAD 2.0. Tokens in [] denote the tokens reconstructed by beam search. More examples are provided in the appendix of the full version [27].

Ground Truth Prompt	Layer	Query Budgets ($\times\|T_{\text{all}}\|$)	RA	Reconstructed Prompt
When was the word "Slavs" used in the national anthem of the Federal Republic of Yugoslavia?	2	1	0.8884	**When was** and **word "Slavs" used in** and **national anthem** for The **Federal Republic** for **Yugoslavia?**
		16	0.9790	**When was** a **word "Slavs" used in the national anthem of the Federal Republic of Yugoslavia?**
	20	1	0.5332	when **was the**[statement] **"Sl** Slaves**" used** In **the national anthem of** The **Federal Republic of Yugoslavia**)?
		16	0.6683	**When was the word** "Sl **Slavs**\n **used**\n **the national anthem of the Federal Republic of Yugoslavia**)?
	L (37)	1	0.1087	**When**[**was**][T][T][T][T][T][T][T][T][riter][riter][T][T][riter][riter][T][executive][executive][T][Wales]'?
		16	0.4191	**When** STOPtrust **word "Slavs**[typew][riter][for][**the**] **national anthem** [bourgeoisie]trust **Federal Republic**[**of**] **Yugoslavia** 950

Table 7: The reconstruction performance of $\mathcal{A}_3$ tested on Phi-3.5 and Llama-3.2.

Metric	Query ($\times\|T_{\text{all}}\|$)	WikiText-2						SQuAD 2.0						Midjourney					
		Phi-3.5			Llama-3.2			Phi-3.5			Llama-3.2			Phi-3.5			Llama-3.2		
		2	18	L (33)	2	10	L (17)	2	18	L (33)	2	10	L (17)	2	18	L (33)	2	10	L (17)
RA	1	0.9208	0.4044	0.1392	0.8747	0.5117	0.1270	0.9809	0.4447	0.2608	0.8735	0.5700	0.3197	0.9955	0.4811	0.2773	0.9061	0.5603	0.3262
	4	**1.0000**	0.4532	0.1714	0.9280	0.6220	0.1813	0.9950	0.5276	0.3346	0.9138	0.6644	0.3606	0.9971	0.5297	0.3066	0.9333	0.6521	0.3391
	8	0.9998	0.5187	0.2206	0.9411	0.6702	0.2388	0.9930	0.5605	0.3778	0.9222	0.6954	0.3818	0.9995	0.5717	0.3634	0.9457	0.6837	0.4018
	16	0.9998	**0.5964**	**0.3063**	**0.9504**	**0.7009**	**0.3343**	**0.9935**	**0.6397**	**0.4235**	**0.9243**	**0.7067**	**0.4400**	**0.9995**	**0.6349**	**0.4221**	**0.9512**	**0.7142**	**0.4800**
CSS	1	0.9865	0.6638	0.6056	0.9832	0.8467	0.6737	0.9953	0.6467	0.5791	0.9699	0.8338	0.7538	0.9993	0.7417	0.6443	0.9791	0.8544	0.7405
	4	**1.0000**	0.7506	0.6200	0.9897	0.8846	0.6739	0.9994	0.6913	0.5883	0.9805	0.8697	0.7579	0.9995	0.7438	0.6713	0.9847	0.8759	0.7537
	8	0.9999	0.7520	0.6297	0.9908	0.9003	0.6857	0.9987	0.7003	0.5941	0.9850	0.8761	0.7614	1.0000	0.7604	0.6908	0.9856	0.8846	0.7634
	16	0.9999	**0.7592**	**0.6552**	**0.9922**	**0.9083**	**0.6960**	**0.9989**	**0.7026**	**0.6142**	**0.9833**	**0.8764**	**0.7818**	**1.0000**	**0.7760**	**0.6977**	**0.9858**	**0.9010**	**0.7655**

Table 8: Price estimation for different attack implementations. Phi-3.5 with $|T_{\text{all}}| = 32064$ and WikiText-2 are used as the evaluation model and dataset, respectively.

Pricing Model	$\mathcal{A}_1$	$\mathcal{A}_2$ ($\delta = 256$)	$\mathcal{A}_3$ of q_b ($\times\|T_{\text{all}}\|$)			
			1	4	8	16
#input tokens	2,396,963	73,152,443	32,064	128,256	256,512	513,024
Gemini 1.5 Pro [1]	\$3.00	\$91.44	\$0.04	\$0.16	\$0.32	\$0.64
GPT-4o [37]	\$5.99	\$182.88	\$0.08	\$0.32	\$0.64	\$1.28

approximately 840 times slower than standard plaintext inference pipelines. A key challenge for future research is to find an effective privacy-efficiency trade-off when employing encryption protocols in distributed LLM inference frameworks.

Differential Privacy. Differential privacy (DP) [16, 18] has been widely adopted in collaborative learning frameworks due to its ability to provide strong privacy guarantees without significantly compromising model inference efficiency, unlike encryption-based methods [23]. A recent study [29] introduces a mechanism Split-N-Denoise (SnD) to protect client embeddings through local differential privacy. In this approach, clients privatize token embeddings

Table 9: Ablation study for $\mathcal{A}_3$ on GPT-2, with query budgets $8 \times |T_{\text{all}}|$ and evaluation metric RA.

Dataset	Phase	Layer (m)				
		2	11	20	29	L (37)
WikiText-2	P1	0.9759	0.8543	0.6091	0.2458	0.0403
	P12	0.9910	0.8551	0.6866	0.5171	0.1652
	P123	0.9932	0.8787	0.7009	0.5858	0.2031
SQuAD 2.0	P1	0.9531	0.8544	0.5737	0.1053	0.0905
	P12	0.9596	0.8659	0.6105	0.3808	0.1864
	P123	0.9635	0.8761	0.6362	0.4473	0.214
Midjourney prompts	P1	0.9782	0.9312	0.6918	0.2011	0.106
	P12	0.9876	0.9345	0.7447	0.6032	0.2731
	P123	0.9936	0.9457	0.7769	0.6695	0.3056

Table 10: Evaluation of the defense mechanism SnD [29]. WikiText-2 is used as the testing dataset.

η of SnD	100			500			∞		
BERTScore (F1)	0.242			0.284			1.000		
GPT-2 Layer	2	20	L (37)	2	20	L (37)	2	20	L (37)
$\mathcal{A}_1$ with SnD	0.048	0.048	0.047	0.073	0.054	0.050	0.997	0.991	0.941

by adding d_χ-privacy noise before transmitting them to the server for further transformations. Although initially designed for "embedding as a service" scenarios, this method can be adapted to distributed LLM inference frameworks without major modifications. To evaluate the effectiveness of SnD against our attacks, we apply $\mathcal{A}_1$ to GPT-2 with the protection of SnD [29]. The privacy parameter η (analogous to ϵ in traditional DP) is varied across $\{100, 500, \infty\}$ as in [29], where ∞ indicates no noise injection. We also use BERTScore (F1) [51] to evaluate the LLM output utility under different levels of noise injection, where scores above 0.8 suggest strong semantic preservation, while scores below 0.5 indicate poor semantic output. Table 10 presents the RA of $\mathcal{A}_1$, from which we see that SnD can significantly degrade the attack performance, rendering it nearly ineffective. The reason is straightforward: injecting DP noise into token embeddings makes the intermediate embeddings corresponding to different tokens nearly indistinguishable, which severely impairs the classification-based attacks. However, *this DP mechanism also significantly diminishes the utility of pretrained LLMs* (BERTScore $\ll 0.5$), as the predicted next token sequences become random and nonsensical, such as ""The "The "The "The". This outcome is expected, given that these LLMs are trained on clean token embeddings, not noisy ones. These findings indicate that DP mechanisms may serve as an effective defense against the proposed attacks, and how to address the privacy-utility trade-off associated with DP mechanisms in distributed LLM inference could be an interesting research problem in future studies.

7 Discussion

Generalization to Deeper LLMs. Although the LLMs used in the experiments have fewer than 36 layers due to hardware limitations, our attacks are expected to generalize well to LLMs with hundreds of layers, as we observed clustered embeddings (the motivation behind our attacks) in deeper LLMs like Llama-3.1-70B (80 layers) [31]. The rationale is that LLMs progressively transform sparse token embeddings into relatively dense linear-layer embeddings through their transformer layers (see Fig. 10). As the number of layers increases, each layer performs a finer-grained transformation, thereby better preserving the cluster structure and enhancing the effectiveness of our attacks.

Evaluation on Proprietary LLMs. Although the proposed attacks are black-box, they have been evaluated only on open-source LLMs and not on proprietary models such as Gemini 1.5 Pro [1] and GPT-4o [37], as these proprietary models have not yet been deployed in distributed LLM frameworks. Future studies are needed to investigate whether the findings obtained from open-source LLMs can generalize to proprietary LLMs.

8 Related Work

Embedding Inversion Attacks. With the rapid advancement of language models, a range of embedding inversion attacks have been developed to explore privacy vulnerabilities in sentence embeddings extracted by these models [12, 22, 34, 35, 38, 41, 44]. These attacks can be broadly categorized into three types: membership inversion [44], keyword inversion [12, 38, 44], and prompt inversion [12, 22, 34, 35, 41, 44]. *Membership inversion* [44] aims to determine whether a user's data was included in the training set that produced specific embeddings. *Keyword inversion* [12, 38, 44] focuses on extracting private identifiers, such as names or dates, from embeddings. Approaches often involve training either a binary classifier [38] or a multi-class classifier [12, 44] using labeled auxiliary data. *Prompt inversion* [12, 22, 34, 35, 41, 44], whose objective aligns closely with our study, seeks to reconstruct input prompts from their corresponding embeddings. These methods are often implemented using either transformer architectures [12, 22, 34, 35] or recurrent neural networks [44]. The main limitations of these methods are that a substantial number of prompt-embedding pairs are required to train the inversion model, and these methods mainly target embeddings produced by shallow models such as GTR-base [22, 34] and BERT [12, 44]. In contrast, we leverage MLPs for prompt reconstruction, which makes our approach more lightweight and adaptable to various types of LLMs. Note that [41] also investigates prompt inversion in distributed LLM frameworks. While their analysis is restricted to white-box and grey-box settings, our attack targets the more challenging black-box scenario.

Other Types of Attacks. Other types of attacks have also been developed to reveal various vulnerabilities in large language models, including training data reconstruction [7], model extraction [21, 48], system prompt extraction [40, 52], and LLM output inversion [11, 49]. Training data reconstruction attacks [7] seek to recover original training samples by carefully crafting prompts that elicit memorized data from LLM outputs. Model extraction attacks [21, 48] aim to replicate the functionality of fine-tuned LLMs by leveraging publicly accessible APIs, effectively stealing proprietary model behavior. System prompt extraction [40, 52] focuses on uncovering proprietary system prompts embedded by developers to control model behavior. Unlike user prompts, system prompts are predefined, guiding the model's responses independently of user inputs and often extracted via black-box API interactions. LLM output inversion [11, 49] focuses on reconstructing user input prompts from LLM outputs based on word statistics and an auxiliary LLM. This approach typically requires multiple queries for the same prompt to accumulate meaningful statistical patterns. Notably, these methods prioritize semantic-level reconstruction, contrasting with our work's focus on token-level reconstruction.

9 Conclusion

In this paper, we examine the privacy risks in distributed LLM inference frameworks via three novel prompt inference attacks designed for diverse deployment scenarios. The proposed attacks are lightweight, context-independent, and highly generalizable, making them both practical and effective. Experimental evaluations reveal that these attacks consistently achieve reconstruction accuracies exceeding 90% in most settings, underscoring the significant privacy vulnerabilities in distributed LLM inference frameworks.

Acknowledgments

Xiaokui Xiao was supported by the Ministry of Education, Singapore, under Tier-2 Grant MOE-000761-01.

References

[1] Google AI. 2024. Pricing. https://ai.google.dev/pricing. Online; accessed 17-December-2024.
[2] Succinctly AI. 2024. midjourney-prompts. https://huggingface.co/datasets/succinctly/midjourney-prompts. Online; accessed 21-November-2024.
[3] Yoshinori Aono, Takuya Hayashi, Le Trieu Phong, and Lihua Wang. 2016. Scalable and Secure Logistic Regression via Homomorphic Encryption. In *Proc. CODASPY 2016*. ACM, 142–144.
[4] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative Inference and Fine-tuning of Large Models. In *Proc. ACL 2023*. 558–568.
[5] Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin A Raffel. 2024. Distributed inference and fine-tuning of large language models over the internet. *NeurIPS* 36 (2024).
[6] Tom B Brown. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* (2020).
[7] Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *USENIX Security 2021*. 2633–2650.
[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proc. NAACL-HLT 2019*. Association for Computational Linguistics, 4171–4186.
[9] evilsocket. 2024. Cake: a Rust framework for distributed inference of large models based on Candle. https://github.com/evilsocket/cake. Online; accessed 22-November-2024.
[10] John Fields, Kevin Chovanec, and Praveen Madiraju. 2024. A survey of text classification with transformers: How wide? how large? how long? how accurate? how expensive? how safe? *IEEE Access* (2024).
[11] Lirong Gao, Ru Peng, Yiming Zhang, and Junbo Zhao. 2024. DORY: Deliberative Prompt Recovery for LLM. In *ACL 2024*. 10614–10632.
[12] Kang Gu, Ehsanul Kabir, Neha Ramsurrun, Soroush Vosoughi, and Shagufta Mehnaz. 2023. Towards Sentence Level Inference Attack Against Pre-trained Language Models. *Proc. Priv. Enhancing Technol.* 2023, 3 (2023), 62–78.
[13] Otkrist Gupta and Ramesh Raskar. 2018. Distributed learning of deep neural network over multiple agents. *J. Netw. Comput. Appl.* 116 (2018), 1–8.
[14] Liang Huang, Kai Zhao, and Mingbo Ma. 2017. When to Finish? Optimal Beam Search for Neural Text Generation (modulo beam size). In *Proc. EMNLP 2017*. 2134–2139.
[15] huggingface.co. 2025. BertGeneration. https://huggingface.co/docs/transformers/en/model_doc/bert-generation. Online; accessed 16-April-2025.
[16] Yangfan Jiang, Xinjian Luo, Yuncheng Wu, Xiaokui Xiao, and Beng Chin Ooi. 2024. Protecting Label Distribution in Cross-Silo Federated Learning. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 113–113.
[17] Yangfan Jiang, Xinjian Luo, Yuncheng Wu, Xiaochen Zhu, Xiaokui Xiao, and Beng Chin Ooi. 2024. On Data Distribution Leakage in Cross-Silo Federated Learning. *IEEE Transactions on Knowledge and Data Engineering* (2024).
[18] Yangfan Jiang, Xinjian Luo, Yin Yang, and Xiaokui Xiao. 2024. Calibrating Noise for Group Privacy in Subsampled Mechanisms. *arXiv preprint arXiv:2408.09943* (2024).
[19] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen, and Binhang Yuan. 2024. HexGen: Generative Inference of Large Language Model over Heterogeneous Environment. In *ICML 2024*.
[20] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *ACM CCS 2020*. ACM, 1575–1590.
[21] Kalpesh Krishna, Gaurav Singh Tomar, Ankur P. Parikh, Nicolas Papernot, and Mohit Iyyer. 2020. Thieves on Sesame Street! Model Extraction of BERT-based APIs. In *ICLR 2020*.
[22] Haoran Li, Mingshi Xu, and Yangqiu Song. 2023. Sentence Embedding Leaks More Information than You Expect: Generative Embedding Inversion Attack to Recover the Whole Sentence. In *ACL 2023*. 14022–14040.
[23] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. 2023. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive* (2023).
[24] Xinjian Luo, Yangfan Jiang, Fei Wei, Yuncheng Wu, Xiaokui Xiao, and Beng Chin Ooi. 2024. Exploring Privacy and Fairness Risks in Sharing Diffusion Models: An Adversarial Perspective. *IEEE Trans. Inf. Forensics Secur.* 19 (2024), 8109–8124.
[25] Xinjian Luo, Yangfan Jiang, and Xiaokui Xiao. 2022. Feature inference attack on shapley values. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 2233–2247.
[26] Xinjian Luo, Yuncheng Wu, Xiaokui Xiao, and Beng Chin Ooi. 2021. Feature inference attack on model predictions in vertical federated learning. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 181–192.
[27] Xinjian Luo, Ting Yu, and Xiaokui Xiao. 2025. Prompt Inference Attack on Distributed Large Language Model Inference Frameworks. *arXiv preprint arXiv:2503.09291* (2025).
[28] Ruilong Ma, Jingyu Wang, Qi Qi, Xiang Yang, Haifeng Sun, Zirui Zhuang, and Jianxin Liao. 2023. Poster: PipeLLM: Pipeline LLM Inference on Heterogeneous Devices with Sequence Slicing. In *Proc. ACM SIGCOMM 2023*. ACM, 1126–1128.
[29] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. 2024. Split-and-Denoise: Protect large language model inference with local differential privacy. In *ICML 2024*.
[30] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843* (2016).
[31] Meta. 2024. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. https://www.llama.com/docs/model-cards-and-prompt-formats/llama3_2. Online; accessed 22-November-2024.
[32] Microsoft. 2024. Discover the New Multi-Lingual, High-Quality Phi-3.5 SLMs. https://techcommunity.microsoft.com/blog/azure-ai-services-blog/discover-the-new-multi-lingual-high-quality-phi-3-5-slms/4225280. Online; accessed 22-November-2024.
[33] Payman Mohassel and Yupeng Zhang. 2017. Secureml: A system for scalable privacy-preserving machine learning. In *SP 2017*. IEEE, 19–38.
[34] John X. Morris, Volodymyr Kuleshov, Vitaly Shmatikov, and Alexander M. Rush. 2023. Text Embeddings Reveal (Almost) As Much As Text. In *Proc. EMNLP 2023*. 12448–12460.
[35] John X. Morris, Wenting Zhao, Justin T. Chiu, Vitaly Shmatikov, and Alexander M. Rush. 2024. Language Model Inversion. In *ICLR 2024*.
[36] Mehran Nasseri, Patrick Brandtner, Robert Zimmermann, Taha Falatouri, Farzaneh Darbanian, and Tobechi Obinwanne. 2023. Applications of large language models (llms) in business analytics–exemplary use cases in data preparation tasks. In *HCI*. Springer, 182–198.
[37] OpenAI. 2024. Pricing. https://openai.com/api/pricing/. Online; accessed 17-December-2024.
[38] Xudong Pan, Mi Zhang, Shouling Ji, and Min Yang. 2020. Privacy Risks of General-Purpose Language Models. In *SP 2020*. IEEE, 1314–1331.
[39] Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. 2021. Unleashing the Tiger: Inference Attacks on Split Learning. In *ACM CCS 2021*. ACM, 2113–2129.
[40] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).
[41] Wenjie Qu, Yuguang Zhou, Yongji Wu, Tingsong Xiao, Binhang Yuan, Yiming Li, and Jiaheng Zhang. 2025. Prompt Inversion Attack against Collaborative Inference of Large Language Models . In *SP 2025*. 1602–1619.
[42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
[43] Pranav Rajpurkar, Robin Jia, and Percy Liang. 2018. Know what you don't know: Unanswerable questions for SQuAD. *arXiv preprint arXiv:1806.03822* (2018).
[44] Congzheng Song and Ananth Raghunathan. 2020. Information Leakage in Embedding Models. In *ACM CCS 2020*. ACM, 377–390.
[45] Jingxuan Tu, Keer Xu, Liulu Yue, Bingyang Ye, Kyeongmin Rim, and James Pustejovsky. 2024. Linguistically Conditioned Semantic Textual Similarity. In *Proc. ACL 2024*. 1161–1172.
[46] A Vaswani. 2017. Attention is all you need. *Advances in Neural Information Processing Systems* (2017).
[47] Bufang Yang, Siyang Jiang, Lilin Xu, Kaiwei Liu, Hai Li, Guoliang Xing, Hongkai Chen, Xiaofan Jiang, and Zhenyu Yan. 2024. Drhouse: An llm-empowered diagnostic reasoning system through harnessing outcomes from sensor data and expert knowledge. *IMWUT 2024* 8, 4 (2024), 1–29.
[48] Santiago Zanella-Beguelin, Shruti Tople, Andrew Paverd, and Boris Köpf. 2021. Grey-box extraction of natural language models. In *ICML*. PMLR, 12278–12286.
[49] Collin Zhang, John X. Morris, and Vitaly Shmatikov. 2024. Extracting Prompts by Inverting LLM Outputs. In *Proc. EMNLP 2024*. 14753–14777.
[50] Mingjin Zhang, Jiannong Cao, Xiaoming Shen, and Zeyang Cui. 2024. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. *arXiv preprint arXiv:2405.14371* (2024).
[51] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *ICLR 2020*.
[52] Yiming Zhang, Nicholas Carlini, and Daphne Ippolito. 2024. Effective prompt extraction from language models. In *First Conference on Language Modeling*.
[53] Junchen Zhao, Yurun Song, Simeng Liu, Ian G. Harris, and Sangeetha Abdu Jyothi. 2023. LinguaLinked: A Distributed Large Language Model Inference System for Mobile Devices. *CoRR* abs/2312.00388 (2023).
[54] Xiaochen Zhu, Xinjian Luo, Yuncheng Wu, Yangfan Jiang, Xiaokui Xiao, and Beng Chin Ooi. 2023. Passive Inference Attacks on Split Learning via Adversarial Regularization. *arXiv preprint arXiv:2310.10483* (2023).